**Université Mohamed Premier**
**Ecole Supérieure de Technologie de Nador**

**Département d'Innovation Technologique et Ingénierie Informatique**

**Filière d'Ingénierie Logicielle et Cybersécurité**

# Rapport de projet de fin d'études

Pour l'obtention du

Diplôme Universitaire de Technologie

Sous thème :

# Conception et réalisation d'un système de Gestion d'une banque de sang : LifeFlow

**Réalisé par :**

**Mr. WALID HAMROUD**

**Mme. IBTIHAL LETRACH**

**Mr. ZAKARYAE MILOUDY**

**Encadré par :**

Mr. Yasser LAMLILI, Professeur à l'ESTN, Encadrant pédagogique

**Co-encadrant :**

Mr. Fouad El Otmani, Professeur à l'ESTN

**Soutenu le 26/03/2026 devant le jury composé de :**

Mr. Redouane ESBAI, Professeur à l'ESTN
Mr. Yasser LAMLILI, Professeur à l'ESTN

Année universitaire : 2025/2026

# Remerciements

Avant tout développement de cette expérience professionnelle, il parait important de commencer ce rapport de Stage de Fin d'Année par des remerciements à ceux et à celles qui nous ont beaucoup apprises, nous ont assistées et accompagnées tout au long de ce stage et à ceux qui ont eu la gentillesse de faire de ce Stage un moment très profitable.

Nous tenons, en particulier, à exprimer notre profonde reconnaissance à M. Yasser Lamlili, notre encadrant et professeur tout au long de notre parcours académique, pour son assistance, sa disponibilité et son accompagnement durant et après la réalisation de ce projet. Grâce à son encadrement rigoureux et à la richesse des connaissances qu'il nous a transmises, il a grandement contribué à l'orientation, à la qualité et à la réussite de ce travail.

Nous tenons également à exprimer notre profonde gratitude à M. Redouane Esbai, Chef de Département, pour la qualité de sa formation, ses directives précieuses et ses conseils pertinents, qui nous ont été d'un appui considérable dans notre démarche. En tant que professeur tout au long de notre parcours académique, il a largement contribué à notre formation, tant sur le plan scientifique que méthodologique, et a joué un rôle déterminant dans l'acquisition de nos connaissances et le développement de nos compétences.

Nous tenons également à remercier l'ensemble des membres du jury qui nous ont fait l'honneur d'accepter d'évaluer et de juger notre travail.

Enfin, nous souhaitons exprimer nos plus sincères remerciements à nos parents et à nos proches pour leur soutien constant, leur patience et leur confiance tout au long de notre parcours académique. Leur amour, leurs encouragements et leurs sacrifices ont été une source inestimable de motivation, de persévérance et de force, contribuant pleinement à la réalisation et à la réussite de ce travail.

# Résumé

Ce projet consiste en la conception et le développement d'une plateforme web complète de gestion de banque de sang nommée LifeFlow, visant à améliorer l'organisation, la traçabilité et l'efficacité des opérations liées au don de sang. La solution proposée permet de centraliser les informations relatives aux donneurs, aux dons, aux stocks sanguins, aux demandes des hôpitaux ainsi qu'aux campagnes de don, tout en assurant un haut niveau de sécurité et de fiabilité.

L'application repose sur une architecture full stack, composée d'un backend développé avec Spring Boot et d'un frontend basé sur Angular. Le backend implémente une architecture en couches (controllers, services, repositories et DTOs) et intègre un système d'authentification et d'autorisation sécurisé basé sur JWT, avec la gestion de trois rôles distincts : *Super Administrateur*, *Administrateur* et *Utilisateur*. Cette gestion des rôles permet de contrôler l'accès aux fonctionnalités selon le profil de chaque utilisateur.

Le système inclut également des fonctionnalités avancées, telles que la gestion des notifications, la gamification à travers l'attribution de badges aux donneurs, ainsi que des tâches automatisées planifiées pour le traitement périodique des données. Des modules d'analyse et de statistiques permettent de suivre l'évolution des dons, l'état des stocks sanguins et les indicateurs globaux, facilitant ainsi la prise de décision.

Côté frontend, l'application Angular offre une interface structurée et évolutive, avec des tableaux de bord adaptés à chaque type d'utilisateur. Elle communique avec le backend via des services HTTP sécurisés, assurant une interaction fluide et cohérente entre les différentes couches de l'application.

En résumé, ce projet propose une solution numérique complète et moderne pour la gestion d'une banque de sang, contribuant à l'optimisation des processus, à l'amélioration de la coordination entre les acteurs concernés et à la valorisation du don de sang à travers des mécanismes innovants.

# Liste des figures

Figure 1 : Méthodologie Scrum .....23
Figure 2 : Diagramme de Gantt .....32
Figure 3: Diagramme use case.....35
Figure 4 : Diagramme de classes .....45
Figure 5: Diagramme de séquences du SuperAdmin.....50
Figure 6 : Diagramme de séquences de l'Admin.....51
Figure 7: Diagramme de séquences du Patient.....53
Figure 8 : Diagramme de séquences de Donneur.....54
Figure 19: Interface d'authentification Mobile.....62
Figure 20: Interface d'accueil Mobile.....63
Figure 21: Interface d'accueil Web.....66
Figure 22 : Interface de gestion des donneurs.....67

# Liste des tableaux

Tableau 1 : Estimation des tâches .....21
Tableau 2: Description use case.....32

# Liste des acronymes

| | |
|---|---|
| UML | Unified Modeling Language |
| SGBDR | Système de Gestion de Base de Données Relationnelle |
| API | Application Programming Interface<br>(Interface de Programmation d'Application) |
| HTTP | HyperText Transfer Protocol |
| JS | JavaScript |
| MVC | Model-View-Controller |
| DB | Database (Base de données) |
| SQL | Structured Query Language |
| UI | User Interface (Interface utilisateur) |
| UX | User Experience (Expérience utilisateur) |
| JWT | JSON Web Token |
| RBAC | Role Based Access Control |
| CSRF | Cross-Site Request Forgery |
| JSON | JavaScript Object Notation |
| HTML | Hypertext Markup Language |

# Table des matières

Remerciements.....3
Résumé .....4
Liste des Figures .....5
Liste des Tableaux.....6
Liste des Acronymes.....7
Introduction Générale.....11
Chapitre I : Analyse des besoins..... 15
1. Étude de l'existant.....16
1.1 Analyse des solutions similaires.....16
1.2. Points communs identifiés..... 17
1.3. Limites des solutions existantes..... 17
1.4. Opportunités d'amélioration..... 17
2.Méthodologie : Gestion de Projet avec la Méthode Agile (SCRUM)..... 18
2.1. Principes de la Méthode Agile (SCRUM)..... 18
2.2. Évenements de Agile Scrum.....19
2.3. Rôles et responsabilités Scrum..... 20
2.4. Outils et techniques utilisés..... 20
2.5. Cycle de développement Agile..... 20
2.6. Avantages de la méthode Agile pour ce projet..... 21
3. Planification de projet : ..... 21
3.1.Estimation des taches..... 21
3.2.Diagramme de Gantt..... 28
4.Spécifications fonctionnelles..... 30
4.1. Diagramme de cas d'utilisation (use case) :..... 30
Chapitre II : Conception et étude technique..... 36
1. Architecture globale..... 36
1.1. Choix de l'architecture..... 36
1.1.1. Outils et techniques utilisés..... 36
1.1.2. Raison d'écarter l'architecture monolithique..... 36
1.1.3. Justification du choix d l'architecture microservices..... 37
1.1.4. Décomposition de LifeFlow en microservices..... 38
1.1.5. Communication entre les microservices et le client mobile..... 39

1.1.6. Impact de l'architecture microservices sur l'application Mobile..... **39**
1.1.7. Limites identifiées et solutions adoptées..... **40**
2. Conception détaillée..... **41**
2.1.Diagrammes de classes..... **41**
2.2.Diagrammes de séquences..... **45**
Chapitre III : Implémentation..... **52**
1. Choix technologiques..... **53**
1.1. Technologies backend :..... **53**
1.2. Base de données:..... **54**
1.3. Environnement et outils : ..... **55**
2.Description des modules développés..... **56**
2.1. Version Mobile :..... **56**
2.2. Version Web :..... **60**
Chapitre IV : Sécurité..... **63**
1. Analyse des risques..... **64**
2. Politique de sécurité..... **65**
3. Mécanismes implémentés..... **66**
3.1. Authentification et gestion des accès..... **66**
3.2. Test de pénétration..... **67**
3.2.1. Tests d'accès aux endpoints sans authentification ..... **69**
3.2.2. Tests d'accès avec token invalide ou expiré..... **69**
3.2.3. Tests de contrôle d'accès et de privilèges..... **69**
3.2.4. Tests automatisés avec OWASP ZAP..... **70**
Chapitre V : Tests et validation .....**71**
1. Tests unitaires ..... **72**
2. Tests d'intégration ..... **72**
3. Tests fonctionnels ..... **72**
4. Tests de performance ..... **73**
5. Résultats et interprétation..... **73**
Chapitre VI : Conclusion générale et perspectives..... **74**
1. Bilan du projet ..... **75**
2. Difficultés rencontrées..... **76**
3. Améliorations futures..... **77**
3.1. Renforcement de la sécurité..... **77**

3.2. Amélioration de la version mobile..... **77**
3.3. Perspectives supplémentaires..... **78**
Conclusion générale ..... **79**
Bibliographie..... **80**
Annexes..... **81**

# Introduction Générale

Le sang est une ressource vitale et irremplaçable. Chaque année, des millions de patients dans le monde dépendent de transfusions sanguines pour survivre à des interventions chirurgicales, à des accidents graves ou à des maladies chroniques. Or, malgré la volonté croissante de nombreux donneurs potentiels, la gestion manuelle et fragmentée des stocks de sang dans les établissements de santé constitue un obstacle majeur à l'efficacité du système transfusionnel. Dans de nombreux pays, y compris en Afrique du Nord et dans de nombreuses régions en développement, les hôpitaux font face à des défis critiques : ruptures fréquentes de stocks sanguins, absence de traçabilité des dons, manque de coordination entre établissements et délais importants dans le traitement des demandes urgentes. Ces lacunes peuvent avoir des conséquences fatales pour les patients nécessitant une transfusion en urgence.

C'est dans ce contexte que s'inscrit le projet LifeFlow, une plateforme numérique complète visant à moderniser et centraliser la gestion des dons de sang. LifeFlow est une application full stack combinant un backend robuste développé avec Spring Boot, une base de données MySQL, et une application mobile Flutter. Elle permet une gestion en temps réel des donneurs, des stocks sanguins, des demandes de transfusion et des campagnes de sensibilisation, en impliquant quatre catégories d'acteurs : le Super Administrateur, l'Administrateur Hôpital, le Donneur et le Patient. La version mobile permet aux utilisateurs d'accéder à toutes les fonctionnalités clés depuis leur smartphone, offrant ainsi réactivité et accessibilité même en dehors des infrastructures hospitalières.

La gestion traditionnelle des dons de sang repose encore, dans de nombreux établissements, sur des fichiers papier, des tableurs non partagés et des communications téléphoniques. Cette approche présente de nombreuses limites qui freinent l'efficacité du système transfusionnel dans son ensemble. L'absence d'un système centralisé empêche le suivi précis des donneurs et des stocks par groupe sanguin, entraînant des ruptures fréquentes ou des excès périssables. Par ailleurs, les hôpitaux ne disposent d'aucun mécanisme pour coordonner ou transférer des ressources sanguines entre établissements en cas de besoin urgent. La collecte et le traitement des demandes de sang des patients restent lents et sujets aux erreurs, pouvant engager le pronostic vital. Enfin, sans système de suivi ni de récompense, les donneurs réguliers ne sont pas fidélisés et les campagnes de don manquent de visibilité. La version

mobile de LifeFlow apporte une solution à ces problèmes, grâce à des notifications push, un suivi en temps réel des stocks et un accès instantané aux informations critiques.

Face à ces constats, la question centrale qui a guidé ce projet est la suivante : **comment concevoir et développer une plateforme numérique intégrée permettant de gérer de manière efficace, sécurisée et collaborative l'ensemble du cycle de vie du don de sang, depuis l'enregistrement du donneur jusqu'à la satisfaction de la demande du patient, en incluant un accès mobile complet et fonctionnel ?**

Le projet LifeFlow poursuit des objectifs clairs articulés autour de deux axes principaux. Sur le plan fonctionnel, il vise à développer une application supportant quatre profils distincts avec des droits spécifiques, à offrir un suivi en temps réel des stocks sanguins par groupe sanguin avec des alertes automatiques en cas de seuil critique, à enregistrer et tracer chaque don de sang, à permettre aux patients de soumettre des demandes urgentes et aux administrateurs de les traiter, à récompenser les donneurs réguliers par un système de badges gamifiés, à créer et diffuser des campagnes de sensibilisation, et à alerter les utilisateurs via des notifications push pour chaque événement important. L'ensemble de ces fonctionnalités est accessible sur mobile, offrant ainsi une réactivité et une disponibilité maximales aux différents acteurs.

Sur le plan technique, LifeFlow ambitionne de mettre en œuvre une architecture microservices, où chaque domaine fonctionnel est encapsulé dans un service indépendant, déployé séparément et exposant ses fonctionnalités via une API REST sécurisée. Il s'agit également d'implémenter une authentification robuste basée sur les tokens JWT avec gestion fine des rôles et des permissions, et de développer une application mobile Flutter communiquant avec les microservices via HTTP et des requêtes RESTful pour assurer un accès rapide et fiable aux données.

Pour mener à bien ce projet dans les délais impartis tout en garantissant une livraison de qualité, nous avons adopté la méthodologie agile Scrum. Cette approche itérative et incrémentale s'est imposée comme le choix naturel pour un projet de cette envergure, où les exigences peuvent évoluer et où la collaboration entre les membres de l'équipe est primordiale. Scrum offre une grande flexibilité, permettant d'intégrer les retours et d'ajuster les priorités à chaque sprint sans remettre en cause l'ensemble du projet. Chaque sprint produit un incrément fonctionnel et testable, réduisant les risques et offrant une visibilité

permanente sur l'avancement. Les cérémonies Scrum — Daily Scrum, Sprint Review et Rétrospective — assurent une communication continue au sein de l'équipe et permettent d'identifier précocement les obstacles pour les résoudre rapidement.

Ce rapport est organisé en cinq chapitres complémentaires qui retracent l'intégralité du cycle de développement du projet LifeFlow.

Le premier chapitre présente une analyse critique des solutions existantes de gestion du don de sang, identifie leurs limites et en déduit les besoins fonctionnels et non fonctionnels ainsi que les contraintes techniques du système LifeFlow, en incluant les diagrammes de cas d'utilisation.

Le deuxième chapitre détaille l'architecture microservices du système, les choix technologiques retenus pour chaque service, et présente l'ensemble des modélisations UML : diagrammes de classes, de séquences, de composants et de cas d'utilisation, ainsi que la conception de la base de données relationnelle.

Le troisième chapitre décrit la réalisation concrète du système en présentant les principaux microservices implémentés et les interfaces mobiles développées pour chacun des quatre rôles, illustrant les choix d'implémentation côté backend (Spring Boot, microservices REST) et côté mobile (Flutter, navigation et gestion des requêtes REST).

Le quatrième chapitre traite des mécanismes de sécurité mis en place : authentification par tokens JWT, gestion des rôles et des permissions, protection des endpoints sensibles, et bonnes pratiques appliquées pour garantir la confidentialité et l'intégrité des données, y compris lors de l'accès via l'application mobile.

Le cinquième chapitre présente la stratégie de tests adoptée (tests unitaires, tests d'intégration, tests fonctionnels), les outils utilisés (Postman, simulateurs mobiles, tests manuels), et les résultats obtenus, concluant par une validation globale du système au regard des exigences initiales et confirmant la fiabilité de la version mobile.

En somme, LifeFlow ambitionne de devenir une référence en matière de gestion numérique du don de sang. En combinant une architecture technique moderne, une expérience utilisateur soignée et accessible sur mobile et web, et une approche méthodologique rigoureuse basée sur Scrum, ce projet illustre comment la technologie peut être mise au

service d’une cause humanitaire d’importance vitale. Les chapitres suivants en décriront progressivement toutes les dimensions, de la conception à la validation.

# Chapitre I : Analyse des besoins

# Chapitre I : Analyse des besoins

## 1. Etude de l'existant

La version mobile de LifeFlow ne se limite pas à une simple adaptation de la version web, mais constitue une véritable évolution du système en rapprochant les fonctionnalités essentielles des utilisateurs. En offrant aux donneurs un accès permanent, simple et rapide à leurs informations et à leurs activités, elle favorise une interaction continue avec la plateforme et renforce leur engagement dans le processus de don.

Grâce à une interface intuitive et à la reprise des principales fonctionnalités du module donneur, l'application mobile permet d'intégrer le don de sang dans le quotidien des utilisateurs, en facilitant la consultation des informations, le suivi des dons et la réception des notifications importantes. Cette accessibilité accrue contribue non seulement à améliorer l'expérience utilisateur, mais également à optimiser la réactivité face aux besoins en sang.

La version mobile joue ainsi un rôle clé dans la modernisation du système LifeFlow, en rendant le processus de don plus fluide, plus interactif et mieux adapté aux usages actuels, tout en participant activement à l'amélioration de l'engagement des donneurs et à l'efficacité globale de la gestion des ressources sanguines.

### 1.1. Analyse des solutions similaires

Plateformes de gestion de banque de sang et don de sang (exemples : Panacea, Clinisys, Wala, Hemosoft)

- **Forces :**
    - Interfaces intuitives pour la gestion des donneurs, des dons, des stocks et des demandes de sang.
    - Suivi automatisé et traçabilité complète des unités de sang, garantissant sécurité et fiabilité.
    - Notifications en temps réel pour alerter les donneurs et le personnel hospitalier.
    - Possibilité de générer des rapports statistiques et dashboards pour les responsables.
- **Faiblesses :**

- Coût élevé des solutions complètes, souvent inaccessibles pour des structures locales ou projets étudiants.
- Peu de fonctionnalités centrées sur l'engagement et la fidélisation des donneurs, comme la gamification ou les récompenses.
- Interfaces parfois complexes pour les utilisateurs non techniques ou pour les patients souhaitant faire une demande de sang.

### 1.2. Points communs identifiés

- Les plateformes existantes proposent généralement une gestion efficace des stocks, des dons et des demandes, mais elles manquent d'intégration fluide entre les différents types d'utilisateurs (SUPER_ADMIN, ADMIN, DONNEUR, PATIENT).
- L'expérience utilisateur (UX) est souvent soignée pour le personnel hospitalier, tandis que les fonctionnalités accessibles aux donneurs ou patients sont limitées.
- Les notifications et le suivi en temps réel jouent un rôle central dans l'efficacité du système et la satisfaction des utilisateurs.

### 1.3. Limites des solutions existantes

- Manque de centralisation : Les super-administrateurs, administrateurs d'hôpitaux, donneurs et patients utilisent souvent des outils séparés, ce qui complique la synchronisation des informations.
- Faible personnalisation : Les solutions disponibles ne permettent pas d'adapter les fonctionnalités aux besoins spécifiques d'un réseau local ou d'une campagne de don ciblée.
- Accessibilité limitée : Les logiciels professionnels sont souvent coûteux et complexes à déployer pour des petites structures ou initiatives locales.

### 1.4. Opportunités d'amélioration

- Développer une plateforme centralisée, intégrant la gestion des utilisateurs, des dons, du stock de sang, des demandes, des hôpitaux et des campagnes.
- Offrir des interfaces intuitives et adaptées à chaque profil (SUPER_ADMIN, ADMIN, DONNEUR, PATIENT).
- Intégrer des fonctionnalités modernes : notifications ciblées, rappels, gamification, et suivi des dons en temps réel.

## 2.Méthodologie : Gestion de Projet avec la Méthode Agile (SCRUM)

La méthode Agile Scrum est une approche itérative et incrémentale pour la gestion de projets, particulièrement adaptée au développement logiciel. Elle met l'accent sur la collaboration, la flexibilité, la rapidité, et la livraison de valeur au client à travers des cycles courts appelés sprints. Chaque sprint produit un livrable potentiellement exploitable, favorisant ainsi des ajustements rapides selon les besoins du client

### 2.1. Principes de Scrum

La méthode Scrum repose sur un ensemble de principes directeurs, considérés comme des extensions des principes agiles. Ces principes définissent la manière dont Scrum est appliqué en pratique afin d'assurer une gestion efficace et flexible du projet.

**Auto-organisation :** Scrum favorise l'autonomie des équipes de développement. Une équipe auto-organisée planifie et gère ses propres tâches, résout les problèmes de manière collective et assume la responsabilité de l'atteinte des objectifs du sprint, sans dépendre d'une autorité hiérarchique externe.

**Limitation du temps (Timeboxing) :** Le timeboxing consiste à fixer une durée déterminée pour les activités Scrum. Cette pratique permet d'optimiser le temps de travail et d'éviter les dérives, notamment lors de la planification du sprint, des réunions quotidiennes et des rétrospectives.

**Priorisation basée sur la valeur :** Les éléments du backlog produit sont priorisés en fonction de leur valeur ajoutée pour l'utilisateur final et les parties prenantes. Cette priorisation est continuellement mise à jour afin de maximiser la valeur du produit livré.

**Développement itératif :** Scrum adopte un développement itératif et incrémental. Le produit est développé par cycles courts (sprints), permettant une amélioration continue et une adaptation rapide aux changements des besoins.

**Collaboration** : La collaboration est un élément central de Scrum. Les différentes cérémonies Scrum, telles que les réunions quotidiennes, les revues de sprint et les rétrospectives, favorisent la communication, la coordination et la résolution rapide des problèmes.

### 2.2. Événements Scrum

**Sprint** : Un sprint est une période définie pour réaliser chaque ensemble de tâches du backlog. Chaque sprint doit avoir la même durée. Deux semaines sont la durée habituelle, mais elle peut varier d'une à quatre semaines selon les besoins de l'équipe et du projet.

**Sprint planning** : Ce processus permet aux équipes d'examiner le backlog produit afin de déterminer comment atteindre les objectifs les plus importants et d'éviter de se disperser. À l'issue de cette analyse, l'équipe comprend mieux le déroulement du développement produit.

**Daily Scrum** : Les réunions quotidiennes Scrum, également appelées standups Scrum, sont des réunions courtes et limitées dans le temps (généralement 15 minutes) au cours desquelles l'équipe Scrum fait le point sur l'avancement et planifie les prochaines 24 heures.

**Product Backlog** : Il s'agit de l'ensemble des fonctionnalités ou des exigences du produit qui doivent être développées. Ces éléments sont priorisés et ajustés au fur et à mesure du projet.

**Product** : À la fin du Sprint, le produit final (ou une partie fonctionnelle de celui-ci) est

livré et prêt à être utilisé ou testé par les parties prenantes.

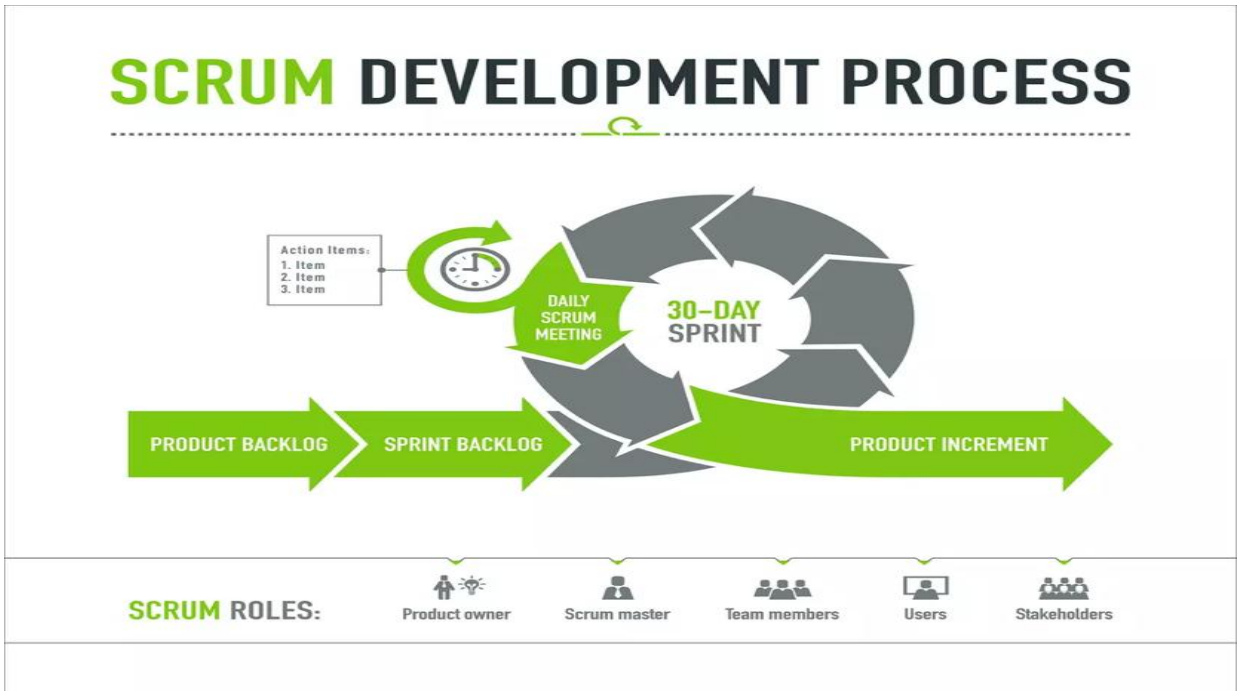



**Figure 1 : Méthodologie Scrum**

### 2.3. Rôles et responsabilités Scrum

**Product Owner** : Le responsable produit a pour mission de maximiser la valeur. Son rôle est de veiller à ce que les intérêts des parties prenantes et des clients soient pris en compte dans la priorisation du backlog produit. Il s'assure également que les livrables répondent à l'évolution des besoins.

**Scrum Master** : Le Scrum Master est souvent décrit comme un leader serviteur : une personne qui responsabilise son équipe plutôt que de la diriger. En tant que guide, il encourage l'équipe à trouver des solutions créatives aux obstacles les plus urgents. Pour ce faire, il pose régulièrement les bonnes questions, celles qui les incitent à trouver des solutions.

**Équipe de développement** : L'équipe de développement est le cerveau du processus. Bien qu'elle travaille avec le soutien du responsable produit et du Scrum Master, elle possède les compétences techniques nécessaires pour concevoir et livrer un excellent produit.

### 2.4. Outils et techniques utilisés

- **Répartition des tâches** : Un document PDF détaillant l'estimation des tâches a été utilisé pour planifier et répartir les responsabilités au sein de l'équipe. Ce document sert de référence principale pour organiser le travail et suivre l'avancement des différentes étapes du projet.
- **Réunions quotidiennes (Daily Stand-ups)** : Synchronisation rapide pour partager les avancements, identifier les blocages, et ajuster les priorités si nécessaires.
- **Prototypage rapide** : Création d'écrans ou de maquettes pour valider les interfaces utilisateur avant leur développement.
- **Intégration continue et tests automatisés** : Garantir la qualité du code à chaque itération grâce à des outils comme GitHub Actions

### 2.5. Cycle de développement Agile

- **Phase d'exploration :**
  - Identification des besoins utilisateurs (administrateur, hôpital, patients, donneurs).
  - Création d'un backlog produit comprenant toutes les fonctionnalités nécessaires.
- **Planification des sprints :**

- Définition des fonctionnalités prioritaires à développer lors du sprint.
- Estimation des efforts nécessaires pour chaque tâche.

➢ **Développement et tests :**

- Conception et développement des modules (par exemple, gestion des demandes ou des stocks).
- Tests unitaires et d'intégration pour assurer la fiabilité des fonctionnalités.

➢ **Validation et livraison :**

- Présentation des résultats aux parties prenantes pour obtenir leurs retours.
- Déploiement des fonctionnalités validées.

➢ **Rétrospective** :

- Analyse des points forts et des axes d'amélioration pour le sprint suivant.

### 2.6. Avantages de la méthode Scrum pour ce projet

- **Livraison de valeur plus rapide et plus fréquente** : Le travail est déployé par petites étapes fonctionnelles à la fin de chaque sprint, permettant ainsi aux clients de constater régulièrement les améliorations.
- **Produit de qualité supérieure** : Scrum fournit le cadre nécessaire à la collaboration et à l'amélioration continue. Des activités comme les rétrospectives et les revues de sprint permettent à l'équipe de réfléchir à ses processus et de les améliorer.
- **Réduction des risques** : Les itérations courtes permettent de détecter rapidement les problèmes et de les corriger sans impact majeur.
- **Amélioration de la communication et de la collaboration** : Les cérémonies Scrum, comme les réunions quotidiennes et les rétrospectives, permettent à tous de rester alignés.

## 3.Planification de projet :

### 3.1 Estimation des taches

| Etape | Tache | Estimation (h/personne) | Risque (30%) | Développeur |
|---|---|---|---|---|
| Etude préalable 54.6h | -Etude de l'existant<br>-Décomposition du projet<br>-Estimation des tâches | 6<br>12<br>12 | 1.8<br>3.6<br>3.6 | Zakaryae+ Ibtihal+Walid |

| | | | | |
|---|---|---|---|---|
| | -Etude de faisabilité | 12 | 3.6 | |
| Etude détaillée<br><br>125.4h | -Conception (UML)<br>• Diagramme d'use case<br>• Diagramme de séquence<br>• Diagramme de classe<br>• Diagramme d'activité<br>• Diagramme de collaboration<br>• Diagramme d'état transition<br>• Diagramme de déploiement<br><br>-Etude technique<br><br>-Maquette statique | **48**<br>6<br>6<br>6<br>6<br>6<br>6<br>6<br><br>**29**<br><br>**20** | 14.4<br><br><br><br><br><br><br><br><br><br><br>8.7<br>6 | <br>Ibtihal<br><br>Zakaryae<br><br>Walid<br><br>Ibtihal<br><br>Ibtihal<br><br>Walid<br><br>Zakaryae<br>Zakaryae+<br>Ibtihal+Walid |
| Réalisation | ❖ **Partie Super Admin**<br><br>**+Ajouter**<br><br>• Admin (Hôpital)<br>• Notification<br>• Campagne<br>• Statistiques<br>• Test unitaire<br><br>**+Modifier**<br><br>• Admin (Hôpital) | **51**<br><br>**9**<br><br>2<br>2<br>2<br>2<br>1<br><br>**11**<br><br>2 | **15.6**<br><br>2.7<br><br><br><br><br><br><br><br>3.3 | <br><br><br><br>Walid |

| | | | | |
|---|---|---|---|---|
| | • Notification | 2 | | Zakaryae |
| | • Campagne | 2 | | |
| | • Statistiques | 2 | | |
| | • Rôle | 2 | | |
| | • Test unitaire | 1 | | |
| | **+Supprimer** | **9** | 2.7 | |
| | • Admin (Hôpital) | 2 | | |
| | • Notification | 2 | | Walid |
| | • Campagne | 2 | | |
| | • Statistiques | 2 | | |
| | • Test unitaire | 1 | | |
| | +**Sélectionner** | **21** | 6.3 | |
| | • Admin (Hôpital) | 2 | | |
| | • Notification | 2 | | Ibtihal |
| | • Campagne | 2 | | |
| | • Statistiques | 2 | | |
| | • Rapport | 2 | | |
| | • Dashboard | 2 | | |
| | • Test unitaire | 1 | | |
| | **Test d'intégration** | **1** | 0.3 | |
| | ❖ **Partie Admin** | **60** | **18.3** | |
| | **+Ajouter** | 13 | 3.9 | |
| | • Patient | 2 | | |
| | • Donneur | 2 | | |
| | • Notification | 2 | | |

| | | | | |
|---|---|---|---|---|
| | • Nouveau don | 2 | | |
| | • Stock | 2 | | |
| | • Demande de sang | 2 | | Zakaryae |
| | • Test unitaire | 1 | | |
| | **+Supprimer** | **11** | 3.3 | |
| | • Patient | 2 | | |
| | • Donneur | 2 | | |
| | • Notification | 2 | | |
| | • Demande | 2 | | |
| | • Stock | 2 | | Ibtihal |
| | • Test unitaire | 1 | | |
| | **+Modifier** | **11** | 3.3 | |
| | • Patient | 2 | | |
| | • Donneur | 2 | | |
| | • Notification | 2 | | |
| | • Demande | 2 | | |
| | • Stock | 2 | | Ibtihal |
| | • Test unitaire | 1 | | |
| | **+Sélectionner** | **25** | 7.5 | |
| | • Liste des donneurs | 2 | | |
| | • Profile des donneurs | 2 | | |
| | • Liste des patients | 2 | | |
| | • Profile des patients | 2 | | |
| | • Historique des dons | 2 | | Walid |
| | • Etat du don | 2 | | |
| | • Liste des dons | 2 | | |
| | • Stock | 2 | | |

| | | | | |
|---|---|---|---|---|
| | • Historique du stock | 2 | | |
| | • Demande de sang | 2 | | |
| | • Notification | 2 | | |
| | • Dashboard | 2 | | |
| | • Test unitaire | 1 | | |
| | **Test d'intégration** | **1** | 0.3 | |
| | ❖ **Partie Patient** | **27** | **8.1** | |
| | **+Ajouter** | 5 | 1.5 | |
| | • Compte | 2 | | |
| | • Demande | 2 | | |
| | • Test unitaire | 1 | | Ibtihal |
| | **+Supprimer** | 5 | 1.5 | |
| | • Compte | 2 | | |
| | • Demande | 2 | | |
| | • Test unitaire | 1 | | Ibtihal |
| | **+Modifier** | 5 | 1.5 | |
| | • Compte | 2 | | |
| | • Demande | 2 | | |
| | • Test unitaire | 1 | | Ibtihal |
| | **+Sélectionner** | 11 | 3.3 | |
| | • Compte | 2 | | |
| | • Etat de la demande | 2 | | |
| | • Notification | 2 | | |

| | | | | |
|---|---|---|---|---|
| | • Dashboard<br>• Disponibilité des hôpitaux<br>• Test unitaire | 2<br>2<br>1 | | Zakaryae |
| | **Test d'intégration** | 1 | 0.3 | |
| | ❖ **Partie Donneur** | **27** | 8.1 | |
| | +**Ajouter** | 3 | 0.9 | |
| | • Compte<br>• Test unitaire | 2<br>1 | | |
| | **+Modifier** | 3 | 0.9 | Walid |
| | • Compte<br>• Test unitaire | 2<br>1 | | |
| | **+Supprimer** | 3 | 0.9 | Walid |
| | • Compte<br>• Test unitaire | 2<br>1 | | |
| | **+Sélectionner** | 17 | 5.1 | Walid |
| | • Compte<br>• Historique des dons<br>• Consulter la date du dernier don<br>• Rappel<br>• Notification<br>• Gamification<br>• Badges | 2<br>2<br>2<br>2<br>2<br>2<br>2 | | Ibtihal |

| | | | | |
|---|---|---|---|---|
| 187h | • Dashboard<br>• Test unitaire<br><br>**Test d'intégration**<br><br>❖ **Partie Authentification**<br>• Recette<br>• Test unitaire<br><br>**Test d'intégration**<br><br>❖ **Documentation**<br>• Recette | 2<br>1<br><br>1<br><br>**12**<br>10<br>1<br><br>1<br><br>**10** | <br><br>0.3<br><br>3.9<br><br><br><br>0.3<br><br>3 | Zakaryae<br><br>Zakaryae+ Ibtihal+Walid |
| Mise en œuvre 8h | ❖ Test de validation | **8** | **2.4** | Zakaryae+ Ibtihal+Walid |
| Total | | 375h | 101.1h | |

**Tableau 1 : Estimation des tâches**

Ce tableau joue un rôle central dans la réussite du projet **LifeFlow**, car il constitue un véritable outil de pilotage, de planification et de suivi du projet.

Tout d'abord, il permet une vision globale et structurée du projet. Chaque étape (étude préalable, étude détaillée, réalisation, tests, mise en œuvre, documentation) est clairement identifiée, ce qui facilite la compréhension de l'ampleur du projet et de son organisation. Pour un projet comme **LifeFlow**, qui implique plusieurs acteurs (Super Admin, Admin, Patient, Donneur), cette structuration est essentielle.

Ensuite, le tableau permet une estimation réaliste des charges de travail. Les tâches sont détaillées et associées à un nombre d'heures par personne, avec une prise en compte du risque (30 %). Cela aide à anticiper les difficultés, à éviter la sous-estimation des efforts nécessaires et à mieux gérer les délais du projet.

Il est également important pour la répartition des responsabilités. Chaque tâche est affectée à un ou plusieurs développeurs (Walid, Ibtihal, Zakaryae), ce qui améliore la coordination de l'équipe, clarifie les rôles et réduit les risques de confusion ou de chevauchement des travaux.

Par ailleurs, ce tableau sert de référence pour le suivi de l'avancement. En comparant les estimations prévues avec le travail réellement réalisé, l'équipe peut mesurer l'état d'avancement du projet **LifeFlow**, détecter les retards éventuels et ajuster les priorités si nécessaire, conformément à l'approche Agile.

Enfin, il renforce la qualité du projet en intégrant explicitement les phases de tests unitaires, tests d'intégration et tests de validation, ainsi que la documentation. Cela garantit que le système **LifeFlow** est fiable, fonctionnel et conforme aux besoins des utilisateurs finaux.

### 3.2 Diagramme de Gantt

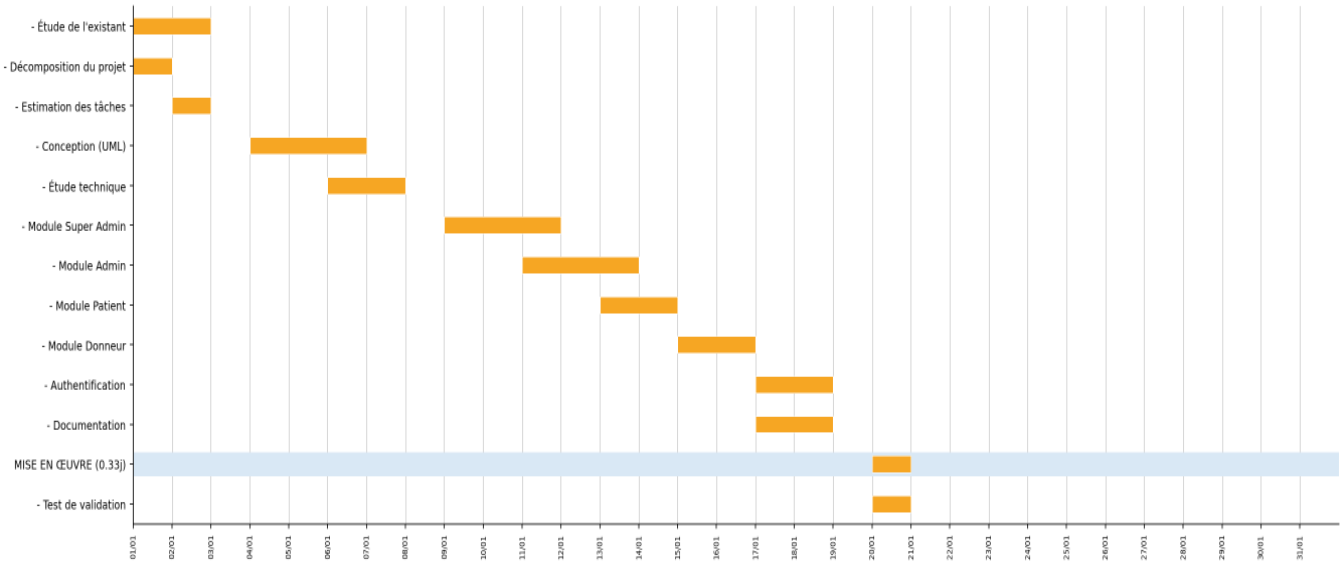



**Figure 2 : Diagramme de Gantt**

Ce diagramme illustre la planification globale du projet sous la forme d'un diagramme de Gantt, un outil de gestion permettant de représenter de manière claire et structurée

l'enchaînement des différentes phases ainsi que leur répartition dans le temps. Il est organisé selon une double lecture : en lignes, les différentes tâches et sous-tâches du projet, et en colonnes, une échelle temporelle exprimée en jours. Les blocs colorés indiquent précisément les périodes durant lesquelles chaque activité est réalisée, ce qui permet d'identifier à la fois la durée, l'ordre d'exécution et les éventuels chevauchements entre les tâches.

Le projet débute par une phase d'étude préalable, estimée à 2,25 jours, qui constitue une étape fondamentale pour la bonne compréhension du contexte et des besoins. Cette phase comprend l'étude de l'existant, visant à analyser les solutions déjà en place et à identifier leurs limites, ainsi que la décomposition du projet en différentes composantes afin de structurer le travail. Elle inclut également l'estimation des tâches, permettant d'évaluer les ressources nécessaires, et l'étude de faisabilité, qui consiste à vérifier la viabilité technique et organisationnelle du projet. Cette première étape joue un rôle déterminant dans la réduction des risques et dans l'orientation des phases suivantes.

La deuxième phase, dite d'étude détaillée, s'étend sur une durée de 5,2 jours et correspond à la phase de conception du système. Elle englobe la modélisation à l'aide des diagrammes UML, qui permettent de formaliser les différents aspects fonctionnels et structurels du système, ainsi que l'étude technique, consacrée au choix des technologies, des outils et des architectures à adopter. Par ailleurs, cette phase inclut la réalisation de maquettes statiques, qui offrent une première représentation visuelle des interfaces utilisateur et facilitent la validation des besoins fonctionnels avant le développement. Cette étape constitue un socle essentiel pour garantir la cohérence et la qualité de la phase de réalisation.

La phase de réalisation, qui s'étale sur 7,79 jours, représente la partie la plus conséquente du projet. Elle est dédiée au développement effectif de l'application et se décline en plusieurs modules, notamment le module Super Admin, le module Admin, le module Patient et le module Donneur. Chaque module est développé sur une période spécifique, avec parfois des chevauchements, traduisant une organisation optimisée du travail et une certaine parallélisation des tâches. Cette phase comprend également la mise en place du système d'authentification, indispensable pour la gestion sécurisée des accès, ainsi que la rédaction de la documentation, qui assure la traçabilité et la compréhension du système développé. L'ensemble de ces activités vise à produire une application fonctionnelle, conforme aux exigences définies en amont.

Enfin, le projet se termine par une phase de mise en œuvre, d'une durée de 0,33 jour, consacrée principalement au test de validation. Cette étape finale permet de vérifier le bon fonctionnement global de l'application, de détecter d'éventuelles anomalies et de s'assurer que toutes les fonctionnalités répondent aux besoins exprimés. Elle constitue une étape essentielle avant la livraison du produit final.

En somme, ce diagramme met en évidence une planification rigoureuse et méthodique du projet, fondée sur une progression logique allant de l'analyse initiale à la validation finale. Il facilite non seulement le suivi de l'avancement des tâches, mais également l'optimisation de la gestion du temps et des ressources, contribuant ainsi au respect des délais et à la réussite globale du projet.

## 4. Spécifications fonctionnelles :

### 4.1. Diagramme de cas d'utilisation (use case) :

Les diagrammes de cas d'utilisation sont utilisés pour modéliser les interactions entre les acteurs et le système en cours de développement. Ils aident à identifier les exigences du système et offrent une vue d'ensemble de la fonctionnalité du système.

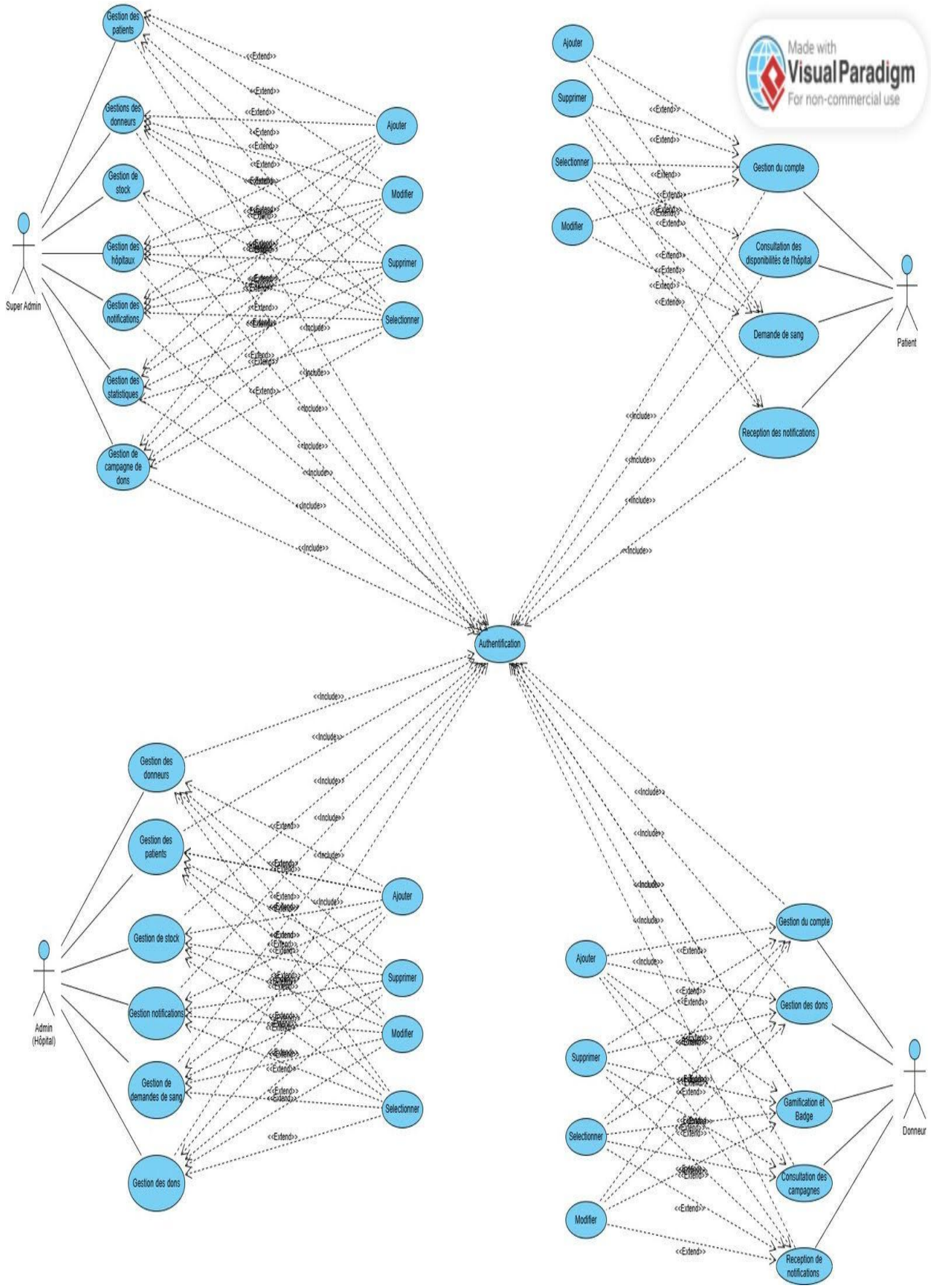



**Figure 3 : Diagramme use case**

## Description

| Acteur | Use case | Description |
|---|---|---|
| **Super admin** | Gestion des patients | Le super administrateur peut gérer l'ensemble des patients du système. |
| | Gestion des donneurs | Le super administrateur peut gérer tous les donneurs enregistrés sur la plateforme. |
| | Gestion des stocks | Le super administrateur peut superviser les stocks de sang de tous les hôpitaux. |
| | Gestion des hôpitaux | Le super administrateur peut ajouter, modifier et supprimer les hôpitaux du système. |
| | Gestion des notifications | Le super administrateur peut gérer les notifications globales du système. |
| | Gestion des statistiques | Le super administrateur peut consulter les statistiques globales liées aux dons, demandes et stocks. |
| | Gestion des campagnes de dons | Le super administrateur peut créer, modifier et gérer les campagnes de dons de sang. |
| **Admin (Hôpital)** | Gestion des donneurs | L'administrateur de l'hôpital peut ajouter, modifier, supprimer et consulter les donneurs. |
| | Gestion des patients | L'administrateur peut gérer les informations des patients. |
| | Gestion du stock | L'administrateur peut consulter, ajouter, modifier et supprimer les stocks de sang. |
| | Gestion des demandes de sang | L'administrateur peut consulter, valider ou rejeter les demandes de sang des patients. |

| | | |
|---|---|---|
| | Gestion des dons | L'administrateur peut suivre et gérer les dons effectués. |
| | Gestion des notifications | L'administrateur peut envoyer et gérer les notifications destinées aux utilisateurs. |
| **Donneur** | Gestion du compte | Le donneur peut créer, consulter et modifier son compte utilisateur. |
| | Gestion des dons | Le donneur peut consulter l'historique de ses dons et s'inscrire pour effectuer un don de sang. |
| | Consultation des campagnes | Le donneur peut consulter les campagnes de dons de sang en cours. |
| | Gamification et badges | Le donneur reçoit des badges et récompenses selon son niveau de participation et le nombre de dons effectués. |
| | Réception des notifications | Le donneur reçoit des notifications liées aux campagnes, dons et rappels. |
| **Patient** | Gestion du compte | Le patient peut créer, consulter et modifier les informations de son compte personnel. |
| | Consultation des disponibilités de l'hôpital | Le patient peut consulter la disponibilité du sang dans les hôpitaux partenaires. |
| | Demande de sang | Le patient peut soumettre une demande de sang en précisant le groupe sanguin et l'hôpital souhaité. |
| | Réception des notifications | Le patient reçoit des notifications concernant l'état de sa demande ou la disponibilité du sang. |
| **Tous les acteurs** | Authentification | Permet à l'utilisateur de se connecter au système en utilisant ses identifiants afin d'accéder aux |

| | | |
|---|---|---|
| | | fonctionnalités correspondant à son rôle. |

**Tableau 2 : Description use case**

# Chapitre II : Conception et étude technique

# Chapitre II : Conception du système

## 1. Architecture globale

### 1.1. Choix de l'architecture

#### 1.1.1. Outils et techniques utilisés

L'architecture microservices constitue un style architectural reposant sur la décomposition d'une application en un ensemble de services indépendants, de taille réduite et spécialisés, chacun étant dédié à un domaine fonctionnel spécifique. Contrairement à l'architecture monolithique, où toutes les fonctionnalités sont regroupées dans un seul artefact, les microservices communiquent entre eux via des API REST. Cette approche offre une meilleure flexibilité en matière de développement, de test, de déploiement et de mise à l'échelle, chaque service pouvant évoluer de manière autonome.

Dans le cadre du projet LifeFlow, cette architecture a été adoptée pour le backend basé sur Spring Boot ainsi que pour le client mobile développé avec Flutter. Ce dernier agit comme un client léger chargé de consommer les différents services exposés par le backend, en établissant des communications via des requêtes HTTP. Flutter permet ainsi de concevoir une interface utilisateur fluide et performante, tout en s'appuyant sur une logique métier centralisée au niveau des microservices.

Ce choix architectural résulte d'une analyse approfondie des besoins du système, notamment en termes de modularité, de maintenabilité et de capacité d'évolution. L'utilisation de Flutter, en combinaison avec une architecture microservices, assure une séparation claire des responsabilités entre les différentes couches de l'application, tout en facilitant son extension future et son adaptation à divers environnements d'exécution.

#### 1.1.2. Raison d'écarter l'architecture monolithique

Avant de retenir l'approche microservices, une réflexion comparative a été menée avec l'architecture monolithique traditionnelle. Dans une architecture monolithique, l'intégralité du système — authentification, gestion des hôpitaux, stocks, dons, demandes et notifications — serait regroupée dans un seul projet Spring Boot. Si cette approche est plus simple à initier, elle présente des limites importantes pour un système aussi critique que LifeFlow.

La première limite est la fragilité : dans un monolithe, une erreur non gérée dans un module peut provoquer l'arrêt complet de l'application, ce qui est inacceptable dans un contexte hospitalier où les vies humaines dépendent de la disponibilité du système. La deuxième limite est la scalabilité rigide : pour absorber un pic de charge sur le module des demandes urgentes, il faudrait dupliquer l'intégralité de l'application, ce qui est coûteux et inefficace. Enfin, la troisième limite est la difficulté de maintenance : plus le système grandit, plus la base de code monolithique devient complexe, les modules se couplant progressivement les uns aux autres, rendant toute modification risquée.

### 1.1.3. Justification du choix d l'architecture microservices

Le choix de l'architecture microservices pour LifeFlow repose sur cinq raisons fondamentales qui ont été analysées et validées lors de la phase de conception.

La première raison est la séparation claire des domaines métier. Le système LifeFlow couvre plusieurs domaines fonctionnels bien distincts : la gestion des donneurs, des hôpitaux, des stocks sanguins, des demandes de transfusion, des notifications et des campagnes. Chaque domaine possède ses propres règles métier et ses propres entités. L'architecture microservices permet de respecter le principe de responsabilité unique au niveau architectural : chaque service ne fait qu'une seule chose, mais il la fait bien. Cette séparation facilite la compréhension du système et réduit les risques de régressions lors des modifications.

La deuxième raison est l'indépendance de déploiement et de mise à l'échelle. Certains services de LifeFlow sont beaucoup plus sollicités que d'autres selon les périodes. Le service de gestion des demandes de sang peut subir une montée en charge brutale lors d'une urgence sanitaire, tandis que le service des campagnes reste peu actif. Avec les microservices, il suffit d'augmenter la capacité uniquement du service concerné, ce qui optimise l'utilisation des ressources et réduit les coûts d'infrastructure.

La troisième raison est la résilience et l'isolation des pannes. L'architecture microservices apporte une isolation naturelle : si le service de notifications tombe en panne, le reste du système continue de fonctionner normalement, notamment la gestion des stocks et le traitement des demandes urgentes. Cette caractéristique est essentielle dans un contexte médical où la disponibilité du système est un impératif absolu.

La quatrième raison est la cohérence avec la méthodologie Scrum. L'adoption de Scrum a naturellement conduit à organiser le travail en sprints thématiques, chacun se concentrant

sur un ou plusieurs services. L'architecture microservices facilite ce découpage : chaque sprint peut livrer un service fonctionnel et testable indépendamment des autres. Plusieurs développeurs peuvent également travailler en parallèle sur des services différents sans conférer sur la même base de code, ce qui améliore la productivité de l'équipe.

La cinquième raison est l'évolutivité et la maintenabilité à long terme. Un système de gestion du don de sang est amené à évoluer : intégration de nouveaux hôpitaux, ajout d'un module de statistiques avancé, connexion à des systèmes externes. L'architecture microservices permet d'ajouter de nouveaux services sans toucher à l'existant, réduisant ainsi le risque de régression. Chaque service peut également être remplacé par une technologie plus adaptée à l'avenir sans refonte globale de l'application.

### 1.1.4. Décomposition de LifeFlow en microservices

La décomposition de LifeFlow en microservices a été guidée par le principe du Domain-Driven Design : chaque service correspond à un contexte métier clairement défini, avec ses propres modèles de données et son propre périmètre fonctionnel. L'architecture LifeFlow se compose de huit microservices indépendants.

L'Auth Service gère l'intégralité du processus d'authentification et d'autorisation. Il est responsable de la création des comptes utilisateurs, de la validation des identifiants lors de la connexion, et de la génération des tokens JWT transmis au client mobile. C'est le premier service invoqué à chaque session utilisateur.

L'Hospital Service prend en charge la gestion complète des établissements de santé enregistrés dans le système. Il permet au Super Administrateur de créer, modifier et supprimer des hôpitaux, et expose la liste des établissements disponibles aux autres services et à l'application mobile.

Le Donation Service gère l'ensemble du cycle de vie des dons de sang, depuis la prise de rendez-vous par le donneur jusqu'à la validation du don par l'administrateur hospitalier. Il maintient l'historique complet des dons associés à chaque donneur.

Le Stock Service assure le suivi en temps réel des stocks sanguins par groupe sanguin dans chaque hôpital. Il détecte automatiquement les seuils critiques et expose des endpoints permettant la mise à jour des quantités, notamment lors de la satisfaction d'une demande de sang.

Le Blood Request Service traite les demandes de transfusion soumises par les patients. Il permet la création, le suivi et la mise à jour du statut des demandes (en attente, acceptée, refusée, satisfaite), et interagit avec le Stock Service pour vérifier la disponibilité du groupe sanguin requis avant de satisfaire une demande.

Le Notification Service centralise l'envoi et la gestion des notifications destinées aux différents utilisateurs. Il est invoqué par les autres services lors d'événements importants tels qu'un stock critique, une demande acceptée ou un don validé, et expose les notifications à l'application mobile via un endpoint dédié.

Le Campaign Service gère les campagnes de sensibilisation au don de sang créées par le Super Administrateur. Il permet la publication, la modification et la suppression des campagnes, qui sont ensuite affichées dans l'application mobile pour tous les utilisateurs.

Enfin, l'API Gateway constitue le point d'entrée unique du système. Il reçoit toutes les requêtes provenant de l'application mobile Android, vérifie les tokens JWT, puis route chaque requête vers le microservice concerné. Cette approche permet de centraliser la sécurité et le routage tout en gardant chaque service métier simple et focussé sur son domaine.

### 1.1.5. Communication entre les microservices et le client mobile

La communication entre l'application mobile Flutter et les microservices repose sur le protocole HTTP/REST, implémenté côté client via des bibliothèques Dart telles que http ou dio. Chaque appel métier de l'application mobile se traduit par une requête HTTP vers l'API Gateway, qui la route vers le microservice approprié. Les échanges de données sont effectués au format JSON, pris en charge nativement par Flutter. Ce mode de communication synchrone convient aux opérations nécessitant une réponse immédiate comme la connexion, la soumission d'une demande ou la mise à jour d'un stock.

### 1.1.6. Impact de l'architecture microservices sur l'application Mobile

L'architecture microservices a directement influencé la conception de l'application mobile Flutter. Chaque domaine fonctionnel correspond à un ensemble cohérent de widgets, services et modèles de données, qui communiquent directement avec les microservices. Cette communication directe permet de simplifier le code client en évitant des couches

intermédiaires, tout en bénéficiant de la granularité et de la modularité des microservices côté serveur.

Cette architecture présente plusieurs avantages concrets pour l'application mobile :

- Si un microservice est en maintenance, seules les fonctionnalités dépendantes de ce service sont affectées, tandis que les autres restent opérationnelles.
- Les microservices peuvent être dimensionnés indépendamment, ce qui garantit des temps de réponse optimaux même lors de fortes charges.
- Chaque microservice peut évoluer ou être mis à jour sans impacter les autres, permettant des mises à jour progressives de l'application mobile sans rupture de service.

### 1.1.7. Limites identifiées et solutions adoptées

L'adoption des microservices a soulevé certaines contraintes lors du développement de LifeFlow. Trois limites principales ont été identifiées :

1. Complexité de la communication inter-services : Pour certaines opérations complexes, plusieurs microservices doivent collaborer pour répondre à une requête par exemple, satisfaire une demande de sang implique le Blood Request Service et le Stock Service. Pour limiter les appels en chaîne et garantir la cohérence, le Stock Service expose un endpoint permettant de vérifier et déduire les stocks de manière atomique, directement invoqué par le Blood Request Service.
2. Gestion de la sécurité : Chaque microservice valide les tokens JWT et gère les permissions selon les rôles. La duplication de la logique de sécurité a été minimisée grâce à des services communs de validation intégrés dans chaque microservice, permettant d'assurer la cohérence des accès et la protection des endpoints sensibles.
3. Complexité de configuration et déploiement : Configurer plusieurs services Spring Boot avec leurs propriétés et ports spécifiques est plus complexe que de gérer un monolithe. Cette charge a été réduite par l'harmonisation des fichiers de configuration et une planification précise des sprints Scrum, chaque sprint étant dédié à un ou deux services, ce qui a permis de structurer efficacement le développement.

En conclusion, l'architecture microservices adoptée pour LifeFlow constitue un choix technique stratégique. Elle permet une scalabilité granulaire adaptée aux besoins critiques

des hôpitaux, garantit la résilience du système et facilite l'évolution des fonctionnalités. Chaque microservice, développé avec Spring Boot et consommé directement par l'application mobile Flutter via des requêtes HTTP REST, représente une brique autonome, testable et évolutive, contribuant à un système robuste et fiable au service d'une cause humanitaire vitale.

## 2. Conception détaillée

### 2.1.Diagramme de classes

Le diagramme de classe représente la structure statique du système de gestion des dons de sang. Il décrit les différentes classes composant le système, leurs attributs, leurs méthodes principales ainsi que les relations entre elles.

Dans cette modélisation, l'ensemble des acteurs (Super Admin, Admin Hôpital, Patient, Donneur) est regroupé dans une seule classe générique appelée « Utilisateur ». La distinction entre les types d'utilisateurs est assurée par un attribut « rôle », conformément au principe du contrôle d'accès basé sur les rôles (RBAC).

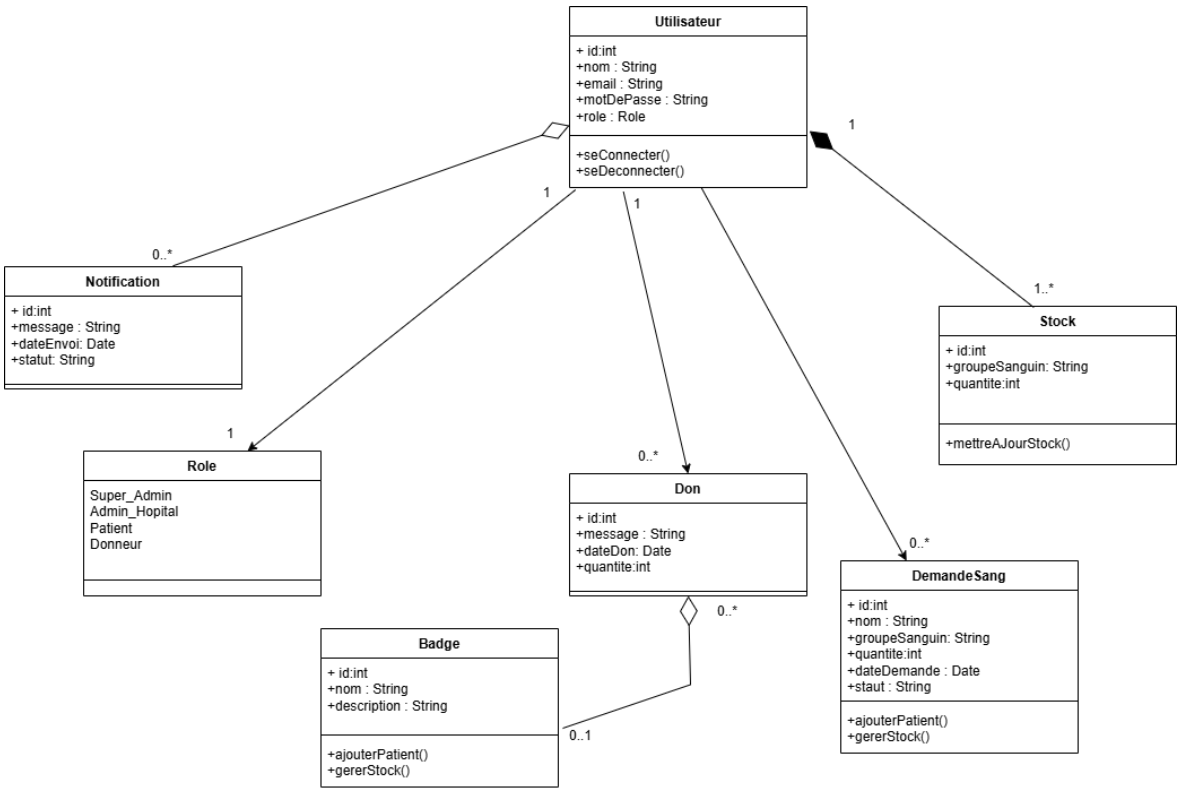



**Figure 4 : Diagramme de classe**

- **Description des classes :**

**1. Classe Utilisateur :**

La classe Utilisateur constitue l'entité centrale du système.

Attributs :

- Id : identifiant unique
- nom : nom complet de l'utilisateur
- email : adresse électronique
- motDePasse : mot de passe sécurisé
- role : type de rôle attribué

Méthodes principales :

- seConnecter()
- seDeconnecter()

L'attribut « role » permet de définir les droits d'accès et les fonctionnalités disponibles pour chaque utilisateur.

**2. Énumération Role :**

L'énumération Role définit les différents rôles possibles dans le système :

- SUPER_ADMIN
- ADMIN_HOPITAL
- PATIENT
- DONNEUR

Cette approche garantit une gestion structurée et sécurisée des autorisations.

**3.Classe Hôpital :**

La classe Hôpital représente les établissements de santé enregistrés dans le système.

Attributs :

- id : identifiant de l'hôpital

- nom : nom de l'établissement
- adresse : localisation
- telephone : numéro de contact

Responsabilités :

- Gérer les patients
- Gérer le stock de sang
- Traiter les demandes de sang

Un hôpital peut être associé à plusieurs réflecteurs du système comme le stock ou les demandes de sang.

**4. Classe DemandeSang :**

La classe DemandeSang représente une demande de sang effectuée par un patient.

Attributs :

- id : identifiant de la demande
- groupeSanguin : type sanguin requis
- quantite : quantité demandée
- dateDemande : date de création
- statut : état de la demande (en attente, validée, traitée)

Chaque demande est associée à un utilisateur ayant le rôle PATIENT et à un hôpital.

**5.Classe Don :**

La classe Don représente un don effectué par un donneur.

Attributs :

- id : identifiant du don
- dateDon : date du don
- quantite : quantité donnée

Un don est associé à un utilisateur ayant le rôle DONNEUR.

**6.Classe Stock**

La classe Stock représente les réserves de sang disponibles dans un hôpital.

Attributs :

- id : identifiant
- groupeSanguin : type sanguin
- quantiteDisponible : quantité disponible

Méthode :

- mettreAJourStock()

Chaque hôpital possède un stock contenant différentes quantités selon les groupes sanguins.

**7.Classe Notification**

La classe Notification permet d'informer les utilisateurs des événements importants.

Attributs :

- id : identifiant
- message : contenu de la notification
- dateEnvoi : date d'envoi
- statut : lu / non lu

Chaque utilisateur peut recevoir plusieurs notifications.

**8.Classe Badge**

La classe Badge est liée au système de gamification.

Attributs :

- id : identifiant
- nom : nom du badge
- description : description du badge

Un badge peut être attribué à un donneur après un ou plusieurs dons.

- **Description des relations :**

Le diagramme met en évidence plusieurs associations importantes :

• Un Utilisateur possède un Role.
• Un Utilisateur peut créer plusieurs DemandeSang.
• Un Utilisateur peut effectuer plusieurs Dons.
• Un Utilisateur peut recevoir plusieurs Notifications.
• Un Don peut donner lieu à l'attribution d'un Badge.
• Un Hôpital gère plusieurs DemandeSang.
• Un Hôpital possède un Stock.

Ces relations illustrent la cohérence fonctionnelle du système et garantissent une organisation claire des données.

### 2.2. Diagrammes de séquences

Le diagramme de séquence global illustre le déroulement dynamique des interactions entre les différents composants du système de gestion des dons de sang. Il met en évidence les échanges entre l'acteur principal « Utilisateur », l'interface web, les différents services métiers et la base de données.

Contrairement à la première version où plusieurs acteurs étaient représentés séparément (Super Admin, Admin Hôpital, Patient, Donneur), la version corrigée regroupe tous les types d'utilisateurs sous un seul acteur générique « Utilisateur ». La distinction des comportements est assurée par un attribut « role », conformément au principe du contrôle d'accès basé sur les rôles (RBAC – Role Based Access Control).

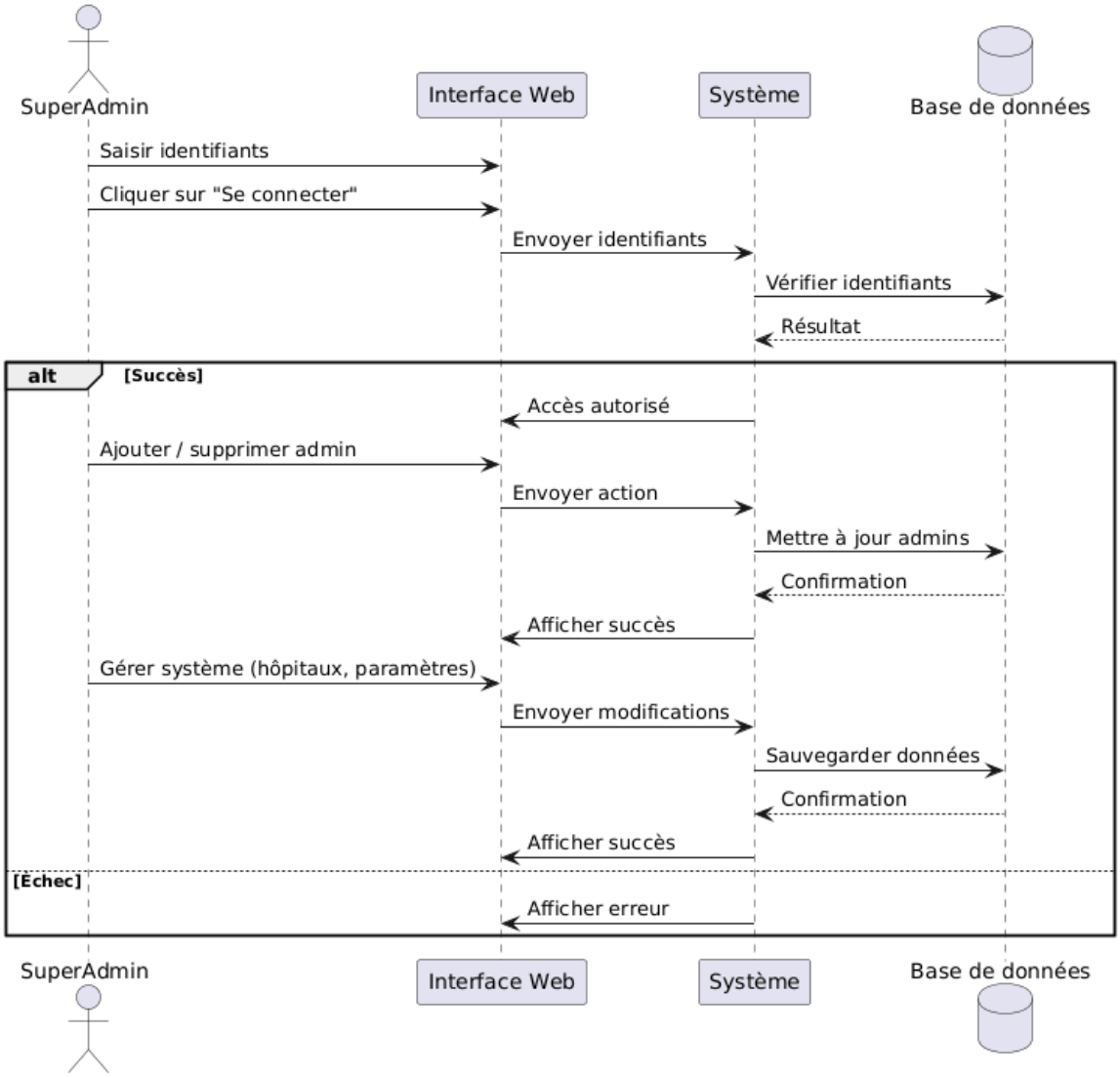



**Figure 5 : Diagramme de séquences du SuperAdmin**

- **Description**

Le diagramme de séquence du SuperAdmin représente les interactions entre cet acteur privilégié et les différents composants du système dans le cadre de la gestion globale de la plateforme. Le processus débute par une phase d'authentification, durant laquelle le SuperAdmin saisit ses identifiants via l'interface web. Ces informations sont ensuite transmises au système applicatif, qui effectue une vérification auprès de la base de données afin de valider l'identité de l'utilisateur.

Une fois l'authentification réussie, le système accorde au SuperAdmin un accès complet aux fonctionnalités stratégiques. Ce dernier dispose de droits étendus lui permettant d'effectuer

des opérations critiques, notamment la gestion des administrateurs (création, modification et suppression des comptes administrateurs). Chaque action initiée depuis l'interface est transmise au système, qui traite la requête et met à jour les données correspondantes dans la base de données. Une confirmation est ensuite renvoyée afin d'assurer la traçabilité et la validation de l'opération.

En outre, le SuperAdmin intervient également dans la gestion globale du système, incluant la configuration des paramètres généraux, la gestion des structures hospitalières, ainsi que la supervision du bon fonctionnement de la plateforme. Ces interactions traduisent son rôle central dans la gouvernance du système.

En cas d'échec d'authentification, le système refuse l'accès et renvoie un message d'erreur à l'interface utilisateur, garantissant ainsi la sécurité et la protection des données sensibles.

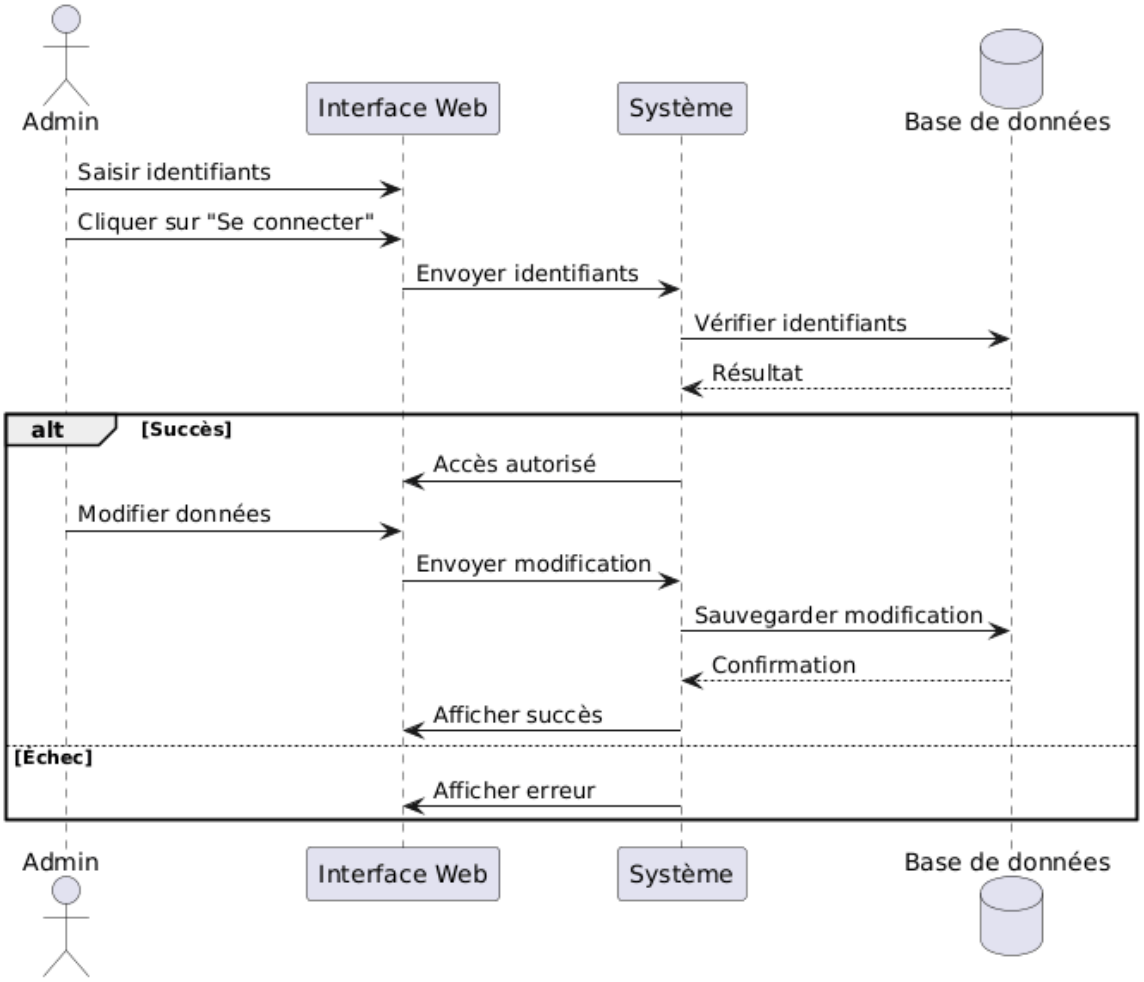



**Figure 6 : Diagramme de séquences de l'Admin**

- **Description**

Le diagramme de séquence de l'Administrateur met en évidence les interactions liées à la gestion opérationnelle des données au sein du système. Le scénario commence par une étape d'authentification où l'administrateur saisit ses identifiants à travers l'interface web. Ces informations sont ensuite envoyées au système, qui procède à leur vérification en consultant la base de données.

Après validation, l'administrateur accède à un ensemble de fonctionnalités lui permettant de gérer les différentes entités du système, telles que les patients, les donneurs, les stocks de sang et les informations relatives aux établissements hospitaliers. Lorsqu'une action est effectuée (modification, ajout ou suppression), celle-ci est transmise au système sous forme de requête. Le système traite ensuite cette requête et met à jour les données correspondantes dans la base de données.

Le retour d'information joue un rôle essentiel dans ce processus : après chaque opération, une confirmation est envoyée à l'interface utilisateur afin d'informer l'administrateur du succès ou de l'échec de l'action. Cela garantit une meilleure traçabilité et une gestion fiable des données.

En cas d'échec de l'authentification, l'accès aux fonctionnalités est refusé, et un message d'erreur est affiché à l'utilisateur, assurant ainsi la sécurité du système.

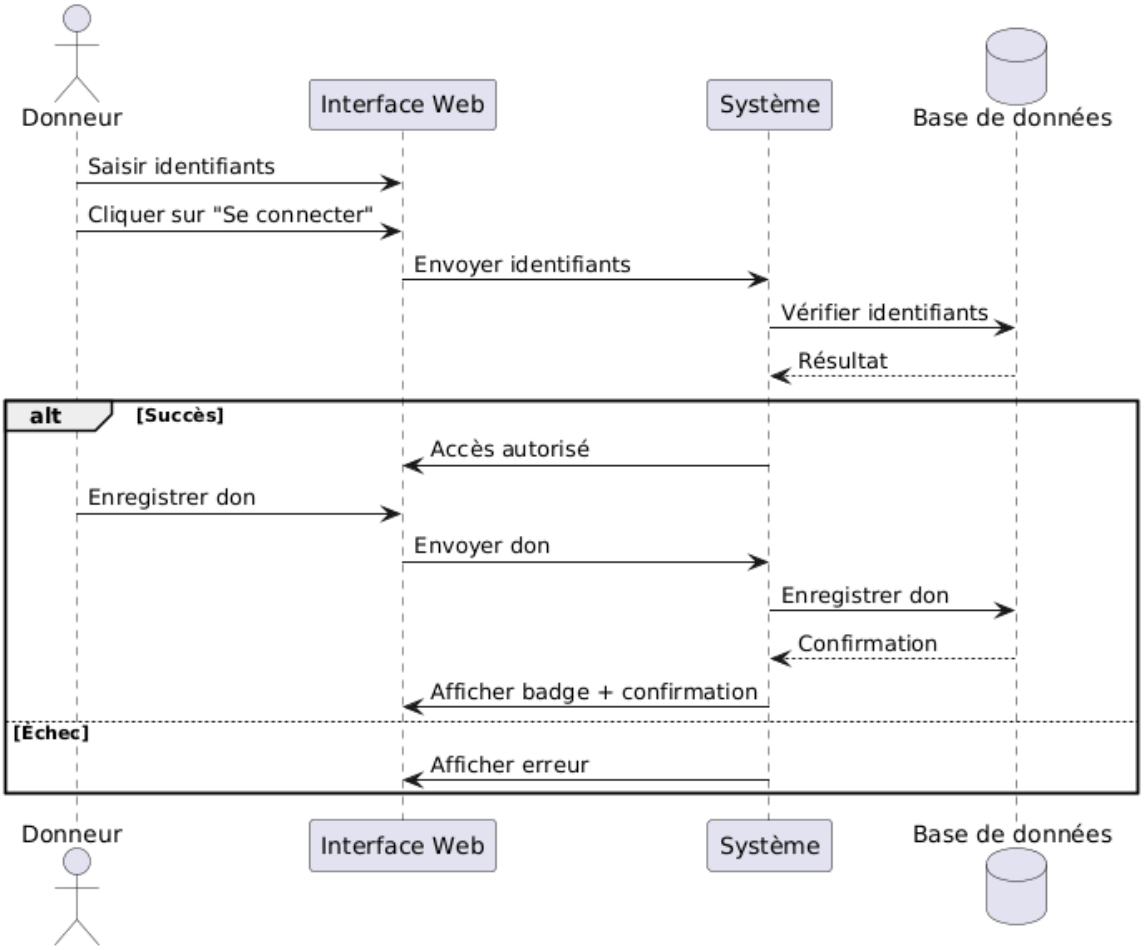



**Figure 7 : Diagramme de séquences du donneur**

- **Description**

Le diagramme de séquence du Donneur met en lumière les interactions associées au processus de don de sang. Comme pour les autres acteurs, le scénario débute par une phase d'authentification au cours de laquelle le donneur saisit ses identifiants via l'interface web. Le système vérifie ensuite ces informations en consultant la base de données.

Une fois authentifié, le donneur peut accéder aux fonctionnalités qui lui permettent d'enregistrer un don de sang. Cette action est effectuée via l'interface utilisateur, puis transmise au système, qui en assure le traitement. Les données relatives au don sont ensuite enregistrées dans la base de données, garantissant leur persistance et leur disponibilité pour un usage ultérieur.

Après l'enregistrement, le système envoie une confirmation à l'interface utilisateur. Dans certains cas, une récompense symbolique, telle qu'un badge, peut être attribuée afin

d'encourager et de valoriser la participation des donneurs. Ce mécanisme peut s'inscrire dans une logique de gamification visant à améliorer l'engagement des utilisateurs.

En cas d'échec de l'authentification, le système bloque l'accès et affiche un message d'erreur, assurant ainsi la sécurité et l'intégrité des données.

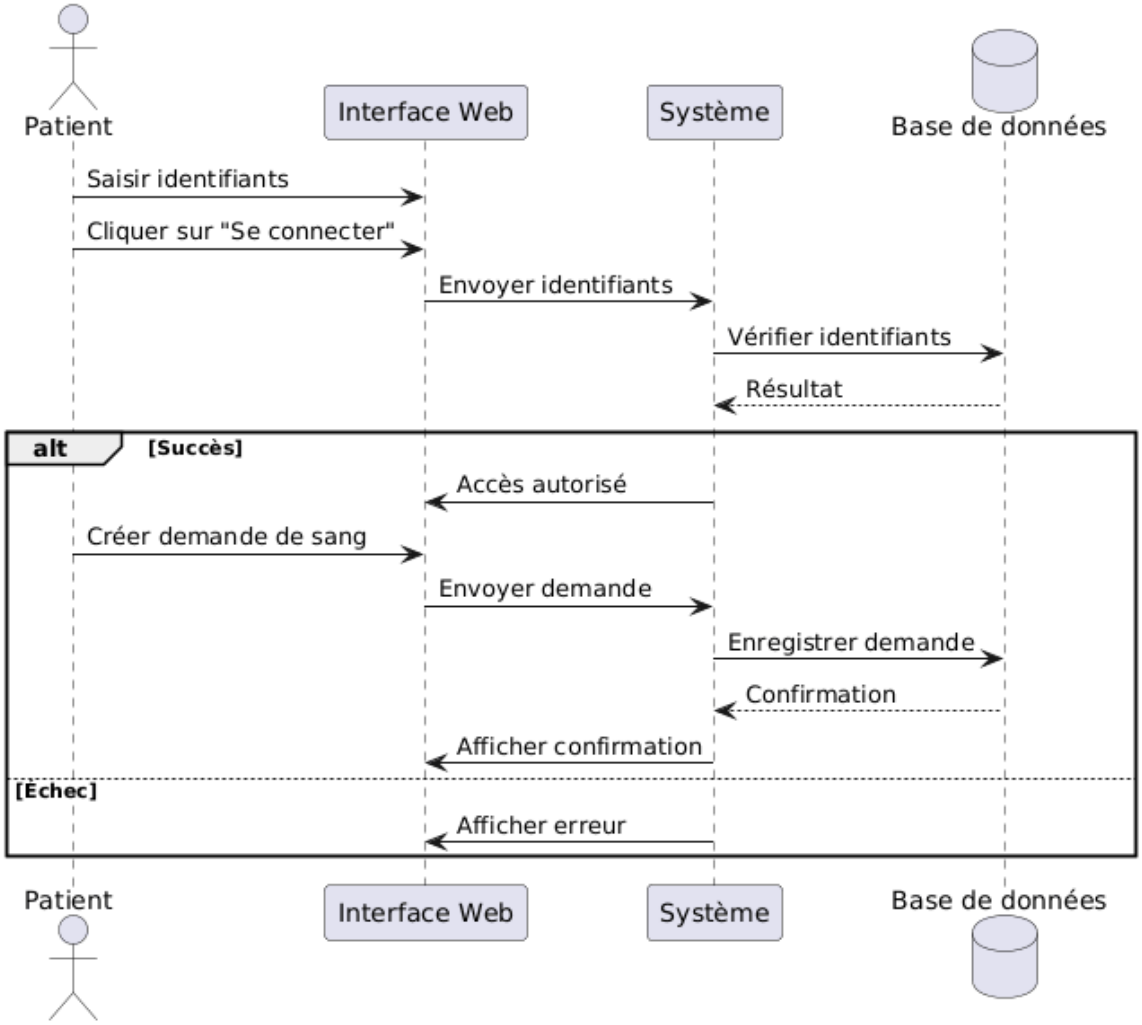



**Figure 8 : Diagramme de séquences du patient**

- **Description**

Le diagramme de séquence du **Patient** illustre les interactions centrées sur la création et la gestion des demandes de sang. Le processus débute par une phase d'authentification, où le patient saisit ses identifiants via l'interface web. Ces informations sont ensuite transmises au système, qui vérifie leur validité en interrogeant la base de données.

Une fois l'accès autorisé, le patient peut utiliser les fonctionnalités qui lui sont dédiées, notamment la création d'une demande de sang. Cette action est initiée via l'interface

utilisateur, puis transmise au système sous forme de requête. Le système traite cette requête en enregistrant les informations correspondantes dans la base de données, ce qui permet de conserver une trace structurée de la demande.

Après l'enregistrement, le système renvoie une confirmation à l'interface web, informant le patient que sa demande a été prise en compte. Cette étape est essentielle pour assurer la transparence et la fiabilité du système du point de vue utilisateur.

En cas d'erreur lors de l'authentification, l'accès est refusé et un message d'erreur est affiché, empêchant toute interaction avec les fonctionnalités du système.

# Chapitre III : Implémentation

# Chapitre III : Implémentation

## 1. Choix technologiques

Le développement du projet LifeFlow a nécessité la sélection rigoureuse d'un ensemble de technologies, de langages et d'outils adaptés à chaque couche du système. Ces choix ont été guidés par des critères de maturité technologique, de compatibilité avec l'architecture microservices adoptée, de productivité de développement et de richesse de l'écosystème.

### 1.1. Technologies backend

#### a. Spring Boot — Framework backend

Framework Java de référence pour les architectures microservices. Chaque service embarque son propre serveur Tomcat et est déployable indépendamment. Son écosystème — Spring Security, Spring Data JPA, Spring Web — couvre l'intégralité des besoins du backend BKS. On utilise Spring Boot pour faciliter et accélérer le développement d'applications Java, notamment les applications web et les API REST, grâce à sa configuration automatique qui évite les réglages complexes, son serveur intégré comme Apache Tomcat qui permet d'exécuter directement l'application sans installation externe, ainsi que son intégration avec l'écosystème Spring Framework offrant des outils puissants pour la gestion des données, de la sécurité et de la logique métier, ce qui permet de produire des applications modernes, structurées, performantes et faciles à maintenir.

#### b. Java — Langage de programmation

Utilisé à la fois pour le backend et l'application mobile, Java garantit une culture technique commune à l'équipe. Son typage statique réduit les erreurs à la compilation et sa maturité assure une compatibilité totale avec l'ensemble des bibliothèques Spring.
Le langage Java est utilisé pour développer des applications robustes, sécurisées et portables, car il repose sur le principe *"write once, run anywhere"* qui permet d'exécuter le même programme sur différents systèmes grâce à la machine virtuelle Java (JVM), tout en offrant une programmation orientée objet qui facilite l'organisation et la réutilisation du code, ainsi qu'un vaste écosystème d'outils et de frameworks comme Spring Boot.

### c. Gradle — Outil de build backend

Gradle est un outil d'automatisation de construction (build tool) utilisé principalement dans le développement logiciel pour gérer, organiser et automatiser les différentes étapes nécessaires à la création d'une application. Il est particulièrement répandu dans le développement d'applications Android, mais il est également utilisé pour des projets Java, Kotlin, Groovy et même d'autres technologies.

On utilise Gradle plutôt que Apache Maven principalement pour sa flexibilité et ses performances, car Gradle utilise un langage basé sur Groovy ou Kotlin qui permet une configuration plus simple et personnalisable que les fichiers XML de Maven, tout en offrant un système de build plus rapide grâce à l'exécution incrémentale et au cache des tâches, ce qui réduit considérablement le temps de compilation dans les grands projets, notamment avec Spring Boot, et le rend ainsi plus adapté aux projets modernes nécessitant rapidité, modularité et optimisation.

## 1.2. Base de données

### a. MariaDB — Système de gestion de base de données

MariaDB est utilisé comme système de gestion de base de données relationnelle pour stocker, organiser et gérer efficacement les données d'une application, tout en offrant de hautes performances, une grande fiabilité et une compatibilité avec MySQL, ce qui facilite son adoption, ainsi qu'un caractère open source qui le rend gratuit et largement utilisé dans les applications web modernes, notamment avec des technologies comme Spring Boot. 1.3. Application mobile

### a. Flutter

Flutter est un framework open-source développé par Google permettant de créer des applications mobiles multiplateformes avec un seul codebase. Cela signifie qu'une seule base de code permet de produire une application fonctionnant à la fois sur Android et iOS, ce qui réduit le temps de développement et facilite la maintenance.

On utilise Flutter parce qu'il permet de développer des applications mobiles performantes et modernes avec un seul code source pour Android et iOS, ce qui réduit le temps et le coût de développement, tout en offrant une interface utilisateur fluide et personnalisable grâce à son moteur graphique et au langage Dart, ainsi qu'un rechargement rapide (hot reload) qui facilite le développement et les tests.

**b. Dart**

Le langage Dart est utilisé pour développer des applications rapides et modernes, notamment avec Flutter, car il est optimisé pour créer des interfaces fluides, offre une syntaxe simple proche de Java et JavaScript, permet un développement rapide grâce au hot reload, et compile en code natif pour garantir de bonnes performances sur mobile, web et desktop.

### 1.3. Environnement et outils

**a. InteliJ IDEA — IDE backend**

Référence pour le développement Java et Spring Boot. Offre une intégration native avec Maven, un support avancé du framework Spring, une détection intelligente des erreurs et des outils de refactorisation qui ont considérablement accéléré le développement des microservices.

On utilise IntelliJ IDEA parce qu'il offre un environnement de développement puissant et intelligent pour Java et les frameworks comme Spring Boot, avec des fonctionnalités avancées comme l'auto-complétion, la détection des erreurs en temps réel, le débogage efficace et l'intégration des outils de build comme Gradle, ce qui permet de développer plus rapidement, avec un code mieux structuré et moins d'erreurs.

**b. Android Studio — IDE mobile**

IDE officiel de Google pour le développement Android. Fournit un émulateur intégré pour tester l'application, un éditeur de mise en page XML visuel et des outils de débogage avancés. Basé sur la plateforme JetBrains, il assure une expérience homogène avec IntelliJ IDEA.

On utilise Android Studio parce qu'il fournit un environnement complet pour développer, tester et déployer des applications Android, avec des outils puissants comme l'éditeur de code intelligent, l'émulateur intégré pour simuler des appareils mobiles, le débogage en temps réel et une intégration parfaite avec Flutter et le langage Dart, ce qui facilite la création d'applications performantes et bien optimisées.

**c. GitHub — Gestion du code source**

Chaque microservice et le projet Android disposent de leur propre dépôt GitHub. La gestion des branches est alignée sur les sprints Scrum, et les pull requests ont permis des revues de code systématiques entre les membres de l'équipe.

On utilise GitHub parce qu'il permet de sauvegarder et gérer le code source de manière sécurisée en ligne, de suivre toutes les modifications grâce au versioning Git, de faciliter le

travail en équipe avec des fonctionnalités comme les branches et les pull requests, et de centraliser le projet pour éviter la perte de code, tout en améliorant la collaboration et l'organisation dans des projets comme ceux développés avec Spring Boot ou Flutter.

**d. Postman — Tests des API REST**

Utilisé pour tester et valider chaque endpoint REST avant son intégration dans l'application mobile. Des collections organisées par microservice couvrent les scénarios nominaux et les cas d'erreur avec tokens JWT, facilitant la détection précoce des régressions entre les sprints. On utilise Postman pour tester, analyser et valider les API de manière simple et rapide, car il permet d'envoyer des requêtes HTTP (GET, POST, PUT, DELETE), de vérifier les réponses du serveur, de déboguer les erreurs et de documenter les API, ce qui est très utile lors du développement de backends avec Spring Boot, afin de s'assurer que les services fonctionnent correctement avant leur intégration avec le frontend.

L'ensemble de ces choix technologiques forme un écosystème cohérent et éprouvé. Spring Boot et Java constituent le socle d'un backend microservices robuste, MariaDB assure la persistance relationnelle avec une base dédiée par service, Flutter offre une expérience mobile performante, tandis que GitHub, Postman, IntelliJ IDEA et Android Studio forment un outillage complet couvrant toutes les phases du cycle de développement.

## 2. Description des modules développés

### 2.1. Version Mobile :

La version mobile de LifeFlow est spécifiquement conçue pour offrir une expérience optimisée aux donneurs de sang, en centralisant toutes les fonctionnalités essentielles pour leur participation et leur suivi au sein du système. L'application permet au donneur de gérer l'ensemble de ses visites, incluant la planification de nouveaux rendez-vous, la modification ou l'annulation de visites déjà programmées, ainsi que le suivi détaillé de l'historique de ses dons précédents. Cette fonctionnalité garantit que le donneur dispose d'une visibilité complète sur ses engagements et facilite la gestion proactive de ses contributions.

En outre, le donneur peut accéder à un système de badges et de reconnaissances, qui matérialise ses participations et ses efforts dans les différentes campagnes de don. Ces badges permettent de suivre l'évolution de son engagement et d'encourager la régularité de ses dons. Parallèlement, l'utilisateur bénéficie d'un module de notifications qui lui transmet en temps réel toutes les informations pertinentes, telles que les rappels de rendez-vous, les annonces des nouvelles campagnes de don ou toute communication émanant des établissements de santé partenaires.

La version mobile intègre également un espace profil complet, où le donneur peut consulter et mettre à jour ses informations personnelles, gérer ses préférences de notification et vérifier ses données de contact et médicales, le tout dans un environnement sécurisé garantissant la confidentialité des données sensibles. L'interface de l'application est conçue pour être intuitive et ergonomique, offrant une navigation fluide et un accès rapide à toutes les fonctionnalités, tout en respectant les standards de sécurité et de protection des données.

Ainsi, cette version mobile de LifeFlow fournit au donneur un outil centralisé et fiable pour suivre ses dons, s'informer sur les campagnes, et rester engagé dans le processus de collecte de sang, contribuant ainsi à l'efficacité et à la réactivité du système de santé.

Parmi les interfaces les plus importantes de l'application, certaines jouent un rôle central dans l'expérience utilisateur du donneur. Elles sont conçues pour assurer une navigation cohérente, intuitive et sécurisée au sein du système. Les captures d'écran présentées ci-dessous illustrent ces interfaces et permettent de visualiser leur organisation et leur ergonomie, offrant ainsi un aperçu concret de l'interaction entre l'utilisateur et l'application.

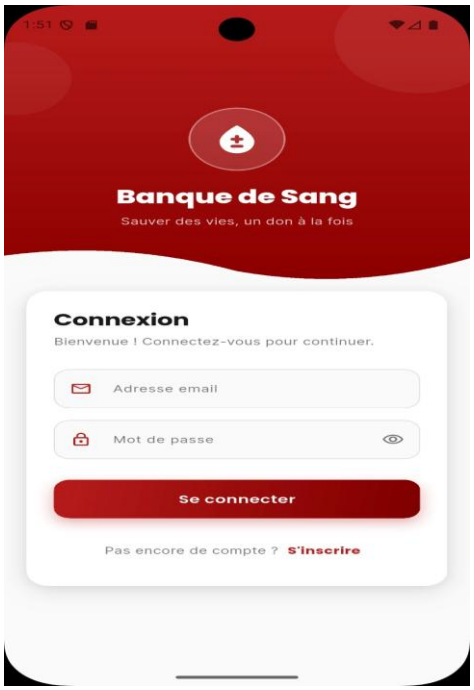



**Figure 9 : Interface d'authentification Mobile**

L'application mobile commence par une interface de connexion sécurisée, où le donneur peut saisir son adresse email et son mot de passe. Une interface d'inscription est également

disponible, permettant la création d'un compte en renseignant les informations personnelles (nom, prénom, email, téléphone) ainsi que les informations de sécurité.

Ces interfaces assurent un accès sécurisé au système grâce à l'authentification basée sur JWT.

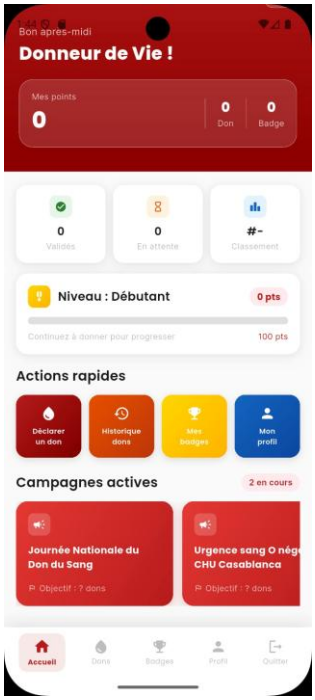



**Figure 10 : Interface d'accueil Mobile**

L'écran d'accueil représente l'interface principale de l'application après authentification. Il constitue un tableau de bord interactif permettant à l'utilisateur de visualiser ses activités, suivre sa progression et accéder rapidement aux différentes fonctionnalités.

La partie supérieure de l'écran affiche un message de bienvenue personnalisé, renforçant l'engagement de l'utilisateur. Cette zone inclut également un en-tête visuellement cohérent avec la charte graphique globale de l'application, notamment par l'utilisation de la couleur rouge.

Un élément central de cette interface est la carte de synthèse des performances de l'utilisateur. Celle-ci présente plusieurs indicateurs clés, notamment :

- Le nombre total de points accumulés,
- Le nombre de dons effectués,
- Le nombre de badges obtenus.

Ces indicateurs s'inscrivent dans une logique de gamification, visant à encourager la participation active des utilisateurs en valorisant leurs contributions.

En complément, des statistiques secondaires sont affichées sous forme de blocs distincts, incluant :

- Le nombre de dons validés,
- Le nombre de dons en attente de validation,
- Le classement de l'utilisateur par rapport à la communauté.

L'interface intègre également un système de progression basé sur des niveaux (par exemple : débutant), accompagné d'une barre visuelle illustrant l'évolution des points. Ce mécanisme renforce la motivation de l'utilisateur en lui offrant des objectifs clairs à atteindre.

Une section dédiée aux actions rapides est mise à disposition afin de simplifier la navigation. Elle comprend plusieurs fonctionnalités essentielles telles que :

- La déclaration d'un don,
- La consultation de l'historique des dons,
- L'accès aux badges obtenus,
- La gestion du profil utilisateur.

Ces actions sont présentées sous forme de boutons visuellement distincts, facilitant leur identification et leur utilisation.

Par ailleurs, l'interface propose une section consacrée aux campagnes actives, permettant d'informer les utilisateurs sur les événements en cours ou les besoins urgents en don de sang. Cette fonctionnalité joue un rôle essentiel dans la sensibilisation et la mobilisation des donneurs.

Enfin, une barre de navigation inférieure est intégrée afin d'assurer un accès rapide aux principales rubriques de l'application, telles que l'accueil, les dons, les badges, le profil et la déconnexion. Cette organisation améliore considérablement l'ergonomie globale et la fluidité de navigation.

Dans l'ensemble, cette interface se distingue par son approche centrée sur l'utilisateur, combinant suivi des performances, accessibilité des fonctionnalités et incitation à

l'engagement, tout en respectant les principes fondamentaux du design d'interfaces mobiles modernes.

### 2.2.Version Web :

La version web de l'application LifeFlow, développée à l'aide des technologies Angular pour la partie frontend et Spring Boot pour le backend, constitue le noyau principal du système. Elle permet d'assurer la gestion complète des activités liées au don de sang en intégrant plusieurs types d'utilisateurs, chacun disposant de fonctionnalités adaptées à son rôle. Cette architecture multi-utilisateur facilite l'organisation, la gestion et la communication entre les différents acteurs impliqués dans le système.

Le super administrateur représente le niveau d'accès le plus élevé au sein de la plateforme. Il joue un rôle essentiel dans la supervision globale du système et dans la gestion des aspects stratégiques. Il est responsable de la gestion des administrateurs, notamment la création, la modification et la suppression de leurs comptes. Il assure également la gestion des campagnes de don de sang ainsi que la diffusion des notifications destinées aux utilisateurs. Son rôle permet de garantir une vision globale du système et d'assurer son bon fonctionnement.

L'administrateur, généralement associé à un hôpital ou un centre de don, assure la gestion opérationnelle des différentes activités. Il est chargé de la gestion des utilisateurs, qu'il s'agisse des donneurs ou des patients, ainsi que du suivi et de la gestion du stock de sang. Il intervient également dans l'attribution des badges aux donneurs, la gestion des notifications et le traitement des demandes de sang. En outre, il participe à la gestion des campagnes de don. Ce rôle est essentiel pour assurer une coordination efficace entre les ressources disponibles et les besoins exprimés.

Le donneur constitue un acteur central du système, puisqu'il participe activement aux campagnes de don de sang. À travers la plateforme, il peut gérer ses visites, consulter l'historique de ses dons et suivre son évolution à travers un système de badges. Il a également accès aux notifications et peut mettre à jour ses informations personnelles via son profil. Ce module vise à améliorer l'engagement des donneurs et à faciliter leur interaction avec le système.

Enfin, le patient est l'utilisateur qui exprime le besoin en sang. Il utilise la plateforme principalement pour effectuer des demandes de sang en fonction de ses besoins médicaux. Il peut également consulter son profil et suivre l'état de ses demandes. Ce module permet d'assurer une meilleure prise en charge des patients en facilitant la communication avec les centres de don et en optimisant la gestion des demandes.

En résumé, la version web de LifeFlow repose sur une organisation claire et structurée des rôles, permettant une gestion efficace, centralisée et collaborative du système de don de sang.

Parmi les interfaces les plus importantes de l'application, certaines jouent un rôle central dans l'expérience utilisateur du donneur. Elles sont conçues pour assurer une navigation cohérente, intuitive et sécurisée au sein du système. Les captures d'écran présentées ci-dessous illustrent ces interfaces et permettent de visualiser leur organisation et leur ergonomie, offrant ainsi un aperçu concret de l'interaction entre l'utilisateur et l'application.

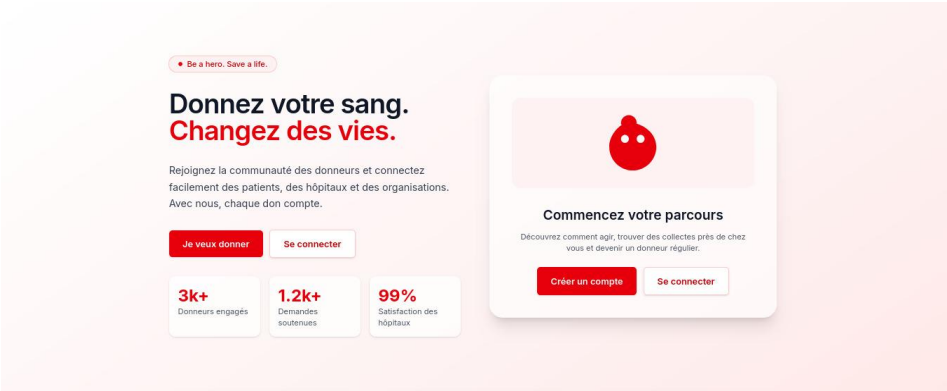



**Figure 11 : Interface d'accueil Web**

Cette interface représente la page d'accueil de la plateforme de don de sang, conçue pour capter l'attention et inciter à l'action. Elle met en avant un message principal fort et engageant, « Donnez votre sang. Changez des vies. », accompagné d'un court texte explicatif qui valorise l'importance du don et l'impact sur la communauté. Deux boutons d'appel à l'action sont clairement visibles : « Je veux donner » et « Se connecter », facilitant l'accès rapide aux principales fonctionnalités. La page inclut également des indicateurs clés (nombre de donneurs engagés, demandes soutenues et taux de satisfaction des hôpitaux) afin de renforcer la crédibilité de la plateforme. Sur la droite, un encadré invite les utilisateurs à commencer leur parcours en créant un compte ou en se connectant, ce qui améliore l'expérience utilisateur en guidant les nouveaux visiteurs de manière intuitive et structurée.

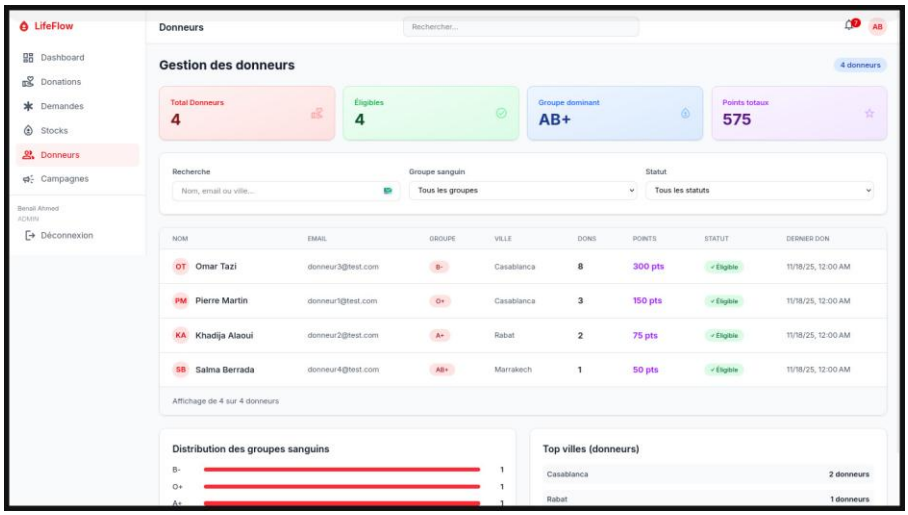



**Figure 12 : Interface de gestion des donneurs**

Cette interface correspond au tableau de bord administratif dédié à la gestion des donneurs. Elle offre une vue globale et structurée des informations essentielles grâce à des indicateurs synthétiques tels que le nombre total de donneurs, le nombre de donneurs éligibles, le groupe sanguin dominant et les points cumulés. Une barre de recherche et des filtres permettent de retrouver facilement des donneurs selon plusieurs critères (nom, groupe sanguin, statut). Le tableau central affiche les données de chaque donneur (nom, email, groupe sanguin, ville, nombre de dons, points et statut), facilitant ainsi le suivi et la gestion. En complément, des sections analytiques comme la distribution des groupes sanguins et les villes les plus actives offrent une dimension décisionnelle, aidant à mieux comprendre les tendances et à optimiser les campagnes de don. L'interface est organisée avec un menu latéral permettant d'accéder aux différentes fonctionnalités du système (dashboard, dons, demandes, stocks, campagnes), assurant une navigation fluide et efficace.

# Chapitre IV : Sécurité

# Chapitre IV : Sécurité

La sécurité constitue un élément central et incontournable du projet LifeFlow. En effet, la plateforme manipule des données médicales particulièrement sensibles, notamment les informations relatives aux profils des donneurs, aux demandes de transfusion ainsi qu'à la gestion des stocks sanguins. De ce fait, elle doit impérativement répondre à des exigences strictes en matière de confidentialité, afin de protéger les données contre tout accès non autorisé, d'intégrité, pour garantir l'exactitude et la fiabilité des informations, et de disponibilité, afin d'assurer un accès continu et fiable au système pour les utilisateurs autorisés.

Dans ce contexte, une attention particulière a été accordée à l'identification et à l'évaluation des risques susceptibles de compromettre la sécurité du système. Sur la base de cette analyse, une politique de sécurité adaptée a été définie afin de mettre en place un cadre cohérent de protection des données et des ressources. Par ailleurs, plusieurs mécanismes techniques ont été implémentés pour renforcer la sécurité globale de la plateforme, tels que les contrôles d'accès, les protocoles de protection des données et les mesures de surveillance.

Enfin, ce chapitre présente également les différents tests de sécurité réalisés dans le but de vérifier l'efficacité des solutions mises en œuvre, d'identifier d'éventuelles vulnérabilités et d'assurer un niveau de sécurité conforme aux exigences du projet.

## 1. Analyse des risques

L'analyse des risques du système LifeFlow a été réalisée en s'appuyant sur les principes de la méthode EBIOS Risk Manager, fondée sur l'expression des besoins et l'identification des objectifs de sécurité, tout en tenant compte des recommandations de la norme ISO/IEC 27005 relative à la gestion des risques de sécurité de l'information. Cette démarche a permis d'identifier de manière structurée les actifs critiques du système, les menaces potentielles ainsi que les vulnérabilités susceptibles d'être exploitées.

Dans le cadre de l'architecture adoptée, reposant sur un backend développé avec Spring Boot, un frontend web en Angular et une application mobile conçue avec Flutter, plusieurs biens essentiels ont été identifiés. Il s'agit notamment des données personnelles des donneurs et des patients stockées dans la base de données, des informations relatives aux stocks sanguins mises à jour en temps réel, des demandes de transfusion avec leur niveau d'urgence,

ainsi que des tokens d'authentification JWT utilisés pour sécuriser les échanges entre les clients (web et mobile) et le serveur backend.

L'analyse des menaces a permis de mettre en évidence plusieurs risques majeurs. Parmi ceux-ci figurent l'accès non autorisé aux données médicales, compromettant leur confidentialité, la modification ou la falsification des informations liées aux stocks ou aux demandes, affectant leur intégrité, ainsi que les attaques par injection de code malveillant via les interfaces Angular, notamment les injections SQL et les attaques XSS. D'autres menaces concernent l'interception des communications entre les applications clientes et le backend Spring Boot, ainsi que les attaques visant à perturber la disponibilité du système, comme les dénis de service.

À l'issue de cette analyse, plusieurs scénarios critiques ont été identifiés et priorisés. Il s'agit notamment de la compromission des tokens JWT pouvant conduire à une usurpation d'identité, des injections SQL ciblant la base de données du backend, des attaques XSS exploitant les formulaires du frontend Angular, ainsi que de la manipulation des droits d'accès via une escalade de privilèges entre les différents rôles du système. Ces risques ont orienté la mise en place de mesures de sécurité adaptées afin de renforcer la protection globale de la plateforme LifeFlow.

## 2. Politique de sécurité

La politique de sécurité du système LifeFlow a été définie afin d'assurer une protection efficace des données médicales et de garantir un usage sécurisé de la plateforme. Elle repose sur les trois principes fondamentaux de la sécurité de l'information, à savoir la confidentialité, l'intégrité et la disponibilité des données.

En matière de confidentialité, l'accès aux informations sensibles est strictement limité aux utilisateurs autorisés. Un mécanisme d'authentification basé sur des tokens JWT a été mis en place afin de sécuriser l'accès au système. De plus, une gestion des rôles a été définie pour différencier les droits d'accès entre les différents types d'utilisateurs, tels que les administrateurs, les agents de santé et les donneurs, garantissant ainsi que chaque utilisateur n'accède qu'aux informations nécessaires à ses fonctions.

Concernant l'intégrité des données, des mécanismes de validation ont été intégrés au niveau du backend Spring Boot ainsi que du frontend Angular afin de vérifier la cohérence et la

validité des données saisies. Les requêtes vers la base de données sont sécurisées pour éviter les attaques par injection, et des contrôles sont effectués pour empêcher toute modification non autorisée des informations critiques, notamment les stocks sanguins et les demandes de transfusion.

Pour assurer la disponibilité du système, des mesures ont été mises en place afin de garantir un accès continu aux services. Cela inclut une gestion appropriée des erreurs, une surveillance du système ainsi qu'une optimisation des performances du backend. L'objectif est de minimiser les interruptions de service et de garantir une utilisation fluide de l'application, aussi bien sur le web que sur mobile.

Par ailleurs, la sécurité des communications est assurée par l'utilisation du protocole HTTPS, permettant de chiffrer les échanges entre les applications clientes (Angular et Flutter) et le serveur Spring Boot. Cette mesure protège les données contre les risques d'interception et d'altération lors de leur transmission.

Enfin, la politique de sécurité de LifeFlow s'inscrit dans une démarche d'amélioration continue, à travers la réalisation de tests réguliers, l'identification des vulnérabilités et la mise à jour des mécanismes de protection, afin de maintenir un niveau de sécurité adapté aux exigences du domaine médical.

## 3. Mécanismes implémentés

Pour garantir la protection des données et la sécurité des utilisateurs, plusieurs mécanismes ont été mis en place dans le système LifeFlow, à la fois sur le backend (Spring Boot) et sur les applications clientes (Angular pour le web et Flutter pour le mobile). Ces mesures ont pour objectif de sécuriser l'accès aux ressources, d'assurer l'authentification des utilisateurs et de limiter les risques liés aux principales attaques informatiques.

### 3.1. Authentification et gestion des accès

L'authentification repose sur l'utilisation de JSON Web Tokens (JWT). Lorsqu'un utilisateur saisit ses identifiants, le backend Spring Boot vérifie leur validité. Si la vérification réussit, un token JWT unique est généré et transmis au client. Ce token doit être inclus dans toutes les requêtes suivantes pour garantir que seules les requêtes provenant d'un utilisateur authentifié sont traitées.

Ce mécanisme présente plusieurs avantages :

- Il permet un contrôle strict des accès aux différentes fonctionnalités du système.
- Il réduit le risque de sessions non sécurisées ou d'usurpation d'identité.
- Il facilite la gestion des sessions sur les applications web et mobiles, sans stocker de session côté serveur.

### 3.2. Tests de Pénétration

Pour garantir la sécurité du système LifeFlow, plusieurs tests approfondis ont été planifiés et réalisés afin de vérifier l'efficacité des mécanismes de sécurité déjà mis en place et d'identifier toute vulnérabilité potentielle avant le déploiement du système. L'objectif principal est de s'assurer que le système protège correctement les données sensibles, empêche tout accès non autorisé et réagit de manière appropriée face aux attaques les plus courantes. Ces tests combinent à la fois un pentesting manuel basique, qui permet de simuler des attaques simples ciblant directement le backend et le frontend, et des scans automatisés utilisant OWASP ZAP, un outil reconnu pour l'analyse de la sécurité des applications web.

Le pentesting manuel basique consiste à simuler des situations réalistes où un utilisateur malveillant pourrait tenter de contourner les mécanismes de sécurité. Parmi les actions testées, on retrouve l'accès aux endpoints sans authentification, l'envoi de tokens JWT invalides ou expirés pour vérifier la robustesse du système face à des tentatives d'usurpation de session. Ce type de tests permet également de contrôler le respect des droits d'accès : chaque rôle utilisateur est testé pour vérifier qu'il ne peut pas accéder aux données ou fonctionnalités qui ne lui sont pas attribuées. Ces vérifications sont essentielles pour garantir que le système ne présente pas de failles évidentes liées à la gestion des sessions ou au contrôle d'accès.

Pour compléter cette approche manuelle, des tests automatisés ont été réalisés avec OWASP ZAP. Cet outil permet d'analyser de manière exhaustive les flux entre le client et le serveur, d'identifier des vulnérabilités courantes et de tester de manière systématique la sécurité des endpoints. ZAP effectue notamment des scans pour détecter des failles comme les injections SQL basiques, le Cross-Site Scripting (XSS), les mauvaises configurations de sécurité, l'exposition d'en-têtes sensibles et d'autres vulnérabilités techniques qui peuvent échapper à un test manuel. De plus, ZAP inclut des fonctionnalités de fuzzing, qui permettent d'envoyer des données aléatoires ou malformées aux endpoints pour vérifier comment le

système réagit face à des entrées inattendues, et d'évaluer la robustesse de la validation côté serveur.

La combinaison de tests manuels et automatisés offre plusieurs avantages :

1. Validation des mécanismes existants : Les tests confirment que les protections mises en place, comme l'authentification JWT et le contrôle d'accès basé sur les rôles, fonctionnent correctement.
2. Détection de vulnérabilités résiduelles : Les scans ZAP peuvent identifier des failles techniques ou des endpoints mal configurés qui n'ont pas été couverts par le pentesting manuel.
3. Évaluation de la résistance du système : L'envoi de requêtes invalides, de tokens expirés ou de données malveillantes permet de vérifier la robustesse du backend face à des situations anormales.
4. Préparation pour des audits futurs : Les résultats des tests fournissent une base de travail pour les audits de sécurité plus avancés, l'amélioration continue du système et l'intégration de mesures supplémentaires, comme la protection contre les attaques DoS ou les injections SQL complexes.

Il est important de noter que même si ces tests sont très utiles, ils présentent certaines limites. Le pentesting manuel est limité aux scénarios simples et ne détecte pas les vulnérabilités logiques complexes ou multi-étapes. Les scans automatisés ZAP sont efficaces pour les vulnérabilités techniques, mais certaines failles métier ou logiques nécessitent des tests spécifiques et une analyse humaine. Par ailleurs, ces tests ne couvrent pas encore les attaques de type DoS ou les tests de charge avancés, qui doivent être réalisés dans un environnement dédié pour éviter tout impact sur le système de production.

En résumé, la stratégie de tests de sécurité mise en place pour LifeFlow combine approche manuelle ciblée et analyse automatisée avec OWASP ZAP, permettant d'identifier et de corriger les failles évidentes et techniques, de vérifier l'efficacité des mécanismes de protection existants et de préparer le système à des audits de sécurité plus approfondis. Cette approche garantit que le système est sécurisé face aux attaques les plus courantes et constitue une base solide pour renforcer la sécurité à mesure que le projet évolue.

### 3.2.1. Tests d'accès aux endpoints sans authentification

- Objectif : Vérifier que les endpoints protégés ne sont accessibles que par des utilisateurs authentifiés.
- Méthode :
  - Les endpoints du backend ont été sollicités directement sans fournir de token JWT.
  - Les requêtes ont été envoyées via un client HTTP (Postman) pour simuler un accès non autorisé.
- Résultat attendu :
  - Le serveur doit rejeter toutes les requêtes non authentifiées.
  - Aucune donnée sensible ne doit être accessible sans authentification.
- Limite : Ce test couvre uniquement l'accès direct aux endpoints et ne détecte pas les vulnérabilités liées aux rôles ou à des injections complexes.

### 3.2.2. Tests d'accès avec token invalide ou expiré

- Objectif : Vérifier que le système rejette les tentatives d'usurpation de session ou d'utilisation de tokens expirés.
- Méthode :
  - Envoi de JWT falsifiés ou expirés vers les endpoints protégés.
  - Simulation de tokens appartenant à d'autres utilisateurs pour tester l'intégrité des droits d'accès.
- Résultat attendu :
  - Les requêtes contenant un token invalide ou expiré doivent être systématiquement rejetées.
  - Le serveur ne doit jamais accorder d'accès à un utilisateur non autorisé.
- Limite : Ce test ne détecte pas les vulnérabilités dans la génération ou la vérification du token côté serveur, qui nécessitent un audit du code.

### 3.2.3. Tests de contrôle d'accès et de privilèges

- Objectif : Vérifier que les utilisateurs ne peuvent accéder qu'aux fonctionnalités et données correspondant à leur rôle.
- Méthode :

    - Utilisation de plusieurs comptes avec des rôles différents (ex. donneur, administrateur).
    - Tentatives d'accès à des endpoints et fonctions non autorisées pour chaque rôle.
- Résultat attendu :
    - Les restrictions de rôle et d'accès doivent fonctionner correctement.
    - Les utilisateurs ne doivent pas voir ni modifier des données qui ne leur appartiennent pas.
- Limite : Ce test ne détecte pas les failles logiques complexes liées à des combinaisons d'actions ou à des workflows spécifiques.

### 3.2.4. Tests automatisés avec OWASP ZAP

- Objectif : Compléter le pentesting manuel avec un scan automatisé pour détecter des vulnérabilités potentielles sur l'application web.
- Méthode :
    - OWASP ZAP a été configuré pour scanner le backend Spring Boot et l'application Angular.
    - L'outil analyse les requêtes et réponses HTTP, identifie les endpoints vulnérables et teste les failles courantes comme :
        - XSS (Cross-Site Scripting)
        - SQL Injection basique
        - Exposition d'en-têtes sensibles
        - Mauvaise configuration de sécurité
    - ZAP effectue également des tests de fuzzing sur les entrées utilisateurs pour détecter des comportements inattendus.
- Résultat attendu :
    - Identification des endpoints vulnérables pour permettre des corrections avant la mise en production.
    - Vérification que les protections JWT sont correctement appliquées.
- Limite : ZAP détecte principalement des vulnérabilités techniques et automatisables. Certaines failles logiques ou liées aux workflows métier nécessitent des tests manuels.

# Chapitre V : Tests et validation

# Chapitre V : Tests et validation

Pour garantir la qualité, la fiabilité et la performance du système LifeFlow, plusieurs types de tests ont été réalisés sur le backend (Spring Boot) et les applications clientes (Angular et Flutter). Ces tests permettent de s'assurer que le système fonctionne correctement, que les composants interagissent bien entre eux et que les performances sont conformes aux attentes.

## 1. Tests unitaires

- Objectif : Vérifier que chaque fonctionnalité ou composant fonctionne correctement individuellement.
- Méthode :
    - Vérification des traitements effectués par le backend, comme la validation des données ou le calcul de certaines informations.
    - Test des actions critiques comme la génération de tokens et la vérification des identifiants.
- Résultat : Chaque élément testé fonctionne comme prévu et ne provoque pas d'erreurs.

## 2. Tests d'intégration

- Objectif : Vérifier que les différents composants du système fonctionnent bien ensemble.
- Méthode :
    - Vérification de la communication entre le backend et la base de données.
    - Test des échanges entre le backend et les applications web et mobile via les API.
- Résultat : Les composants interagissent correctement et les données sont traitées et transmises comme prévu.

## 3. Tests fonctionnels

- Objectif : S'assurer que le système remplit les fonctionnalités définies pour les utilisateurs.
- Méthode :
    - Simulation des actions des utilisateurs, comme la connexion, l'ajout, la modification ou la suppression de données.

    - Vérification que les utilisateurs ne peuvent accéder qu'aux fonctionnalités correspondant à leur rôle.
- Résultat : Toutes les fonctionnalités principales sont opérationnelles et répondent aux besoins.

## 4. Tests de performance

- Objectif : Vérifier que le système reste rapide et stable, même avec plusieurs utilisateurs ou de grandes quantités de données.
- Méthode :
    - Vérification du temps de réponse lors de l'accès aux données et aux fonctionnalités.
    - Observation de la réactivité des applications web et mobile pendant plusieurs actions simultanées.
- Résultat : Le système reste fluide et les temps de réponse sont satisfaisants.

## 5. Résultats et interprétation

- Les tests montrent que LifeFlow est fiable, fonctionnel et stable.
- Les principales fonctionnalités, la gestion des utilisateurs, l'accès aux données et la sécurité des informations sont conformes aux attentes.
- Ces vérifications ont permis de détecter et corriger les erreurs avant le déploiement, garantissant une utilisation sûre et efficace du système.

# Chapitre VI :

# Conclusion générale et perspectives

# Chapitre VI : Conclusion générale et perspectives

## 1. Bilan du projet

Le projet LifeFlow a permis de concevoir, développer et déployer un système de gestion complet, à la fois efficace, sécurisé et accessible sur différentes plateformes, incluant le web et le mobile. Ce système a été pensé pour répondre aux besoins critiques des établissements médicaux en matière de gestion des banques de sang, en garantissant la fiabilité, la traçabilité et la sécurité des données.

Le backend, développé avec Spring Boot, constitue le cœur du système. Il assure la gestion complète des données relatives aux utilisateurs, aux dons de sang, aux stocks et aux transferts. Chaque requête est traitée de manière sécurisée grâce à l'utilisation de tokens JWT, qui garantissent que seules les personnes authentifiées peuvent accéder aux informations sensibles. De plus, la configuration de Spring Security renforce cette protection en assurant un contrôle strict des accès selon les rôles des utilisateurs, empêchant toute action non autorisée. Le backend est également structuré pour être modulable et évolutif, permettant d'ajouter facilement de nouvelles fonctionnalités ou d'adapter le système aux besoins futurs.

Les applications clientes, développées respectivement avec Angular pour le web et Flutter pour le mobile, offrent une interface intuitive, fluide et réactive. Ces applications permettent aux utilisateurs d'accéder facilement aux fonctionnalités principales, comme la consultation des stocks de sang, l'enregistrement des dons, la planification des transfusions et la génération de rapports. L'interface a été conçue pour être ergonomique et simple, facilitant la navigation et réduisant le risque d'erreurs lors de la saisie des données.

Tout au long du développement, différents types de tests ont été réalisés pour s'assurer que le système fonctionne correctement. Les tests fonctionnels ont permis de vérifier que toutes les fonctionnalités prévues sont opérationnelles et répondent aux besoins définis dans le cahier des charges. Les tests d'intégration ont validé que les composants du backend, la base de données et les applications clientes interagissent correctement et de manière cohérente. Les tests de performance ont confirmé que le système reste réactif et stable même lorsque plusieurs utilisateurs effectuent des actions simultanément, garantissant ainsi une expérience utilisateur fluide et fiable.

En parallèle, des tests de sécurité et un pentesting basique, complétés par des scans automatisés avec OWASP ZAP, ont été réalisés pour identifier les vulnérabilités et vérifier l'efficacité des mécanismes de protection. Ces tests ont permis de confirmer la robustesse de l'authentification par JWT, l'efficacité des protections contre les attaques CSRF et le respect strict des droits d'accès selon les rôles des utilisateurs. Ils ont également permis de détecter des points à renforcer, ouvrant la voie à des améliorations futures en matière de sécurité et de prévention des attaques plus complexes.

En somme, le projet a permis de créer un outil complet et opérationnel pour la gestion des banques de sang, qui est à la fois fiable, sécurisé, performant et facile à utiliser. Il répond pleinement aux besoins des utilisateurs et constitue une base solide pour de futures évolutions, qu'il s'agisse d'ajout de fonctionnalités, d'amélioration des performances ou du renforcement de la sécurité. Le projet LifeFlow illustre ainsi la réussite de la conception et de la mise en œuvre d'un système de gestion moderne, capable de soutenir efficacement les opérations critiques dans le domaine médical.

## 2. Difficultés rencontrées

Plusieurs difficultés ont été rencontrées au cours du projet :

- Sécurité et authentification : La mise en place des tokens JWT et de la protection contre certaines attaques comme
- Intégration multi-plateformes : Assurer la communication fluide entre le backend et les applications web et mobile a demandé de tester et de corriger plusieurs problèmes liés aux API et aux formats de données.
- Tests et vérifications : L'absence d'outils automatisés pour certains tests a rendu certaines vérifications plus longues, nécessitant des tests manuels et une attention particulière pour valider toutes les fonctionnalités.
- Gestion des erreurs et cas limites : Certains scénarios particuliers, comme des entrées incorrectes ou des utilisateurs non autorisés, ont nécessité des ajustements pour garantir la robustesse du système.

Malgré ces difficultés, toutes ont été surmontées grâce à des tests réguliers, une documentation soignée et une approche progressive du développement.

## 3. Améliorations futures

Pour renforcer et faire évoluer le système LifeFlow, plusieurs axes d'amélioration ont été identifiés. Ces améliorations visent principalement à renforcer la sécurité et à développer une version mobile plus complète et performante, tout en assurant la stabilité et la fiabilité du système.

### 3.1. Renforcement de la sécurité

La protection des données sensibles est un enjeu majeur, surtout dans un système manipulant des informations médicales. Les améliorations envisagées incluent :

- Renforcement des mécanismes d'authentification et de contrôle d'accès : Même si le système utilise déjà les tokens JWT et les rôles utilisateur, il est possible d'ajouter des vérifications supplémentaires et des mécanismes de renouvellement automatique des tokens pour améliorer la sécurité des sessions.
- Amélioration de la protection contre les attaques avancées : Bien que le backend soit protégé contre les attaques classiques comme XSS ou les injections SQL basiques, des mesures supplémentaires pourraient être mises en place pour prévenir les attaques plus complexes ou ciblées.
- Surveillance et audits réguliers : Mettre en place un suivi continu et des outils d'alerte pour détecter toute activité suspecte ou vulnérabilité émergente, permettant de réagir rapidement avant que les données ne soient compromises.

### 3.2. Amélioration de la version mobile

Le développement et l'amélioration de l'application mobile Flutter sont essentiels pour rendre le système plus accessible et pratique :

- Optimisation de l'expérience utilisateur : Améliorer l'interface mobile pour qu'elle soit plus intuitive et réactive, avec une navigation fluide et des affichages adaptés aux différents formats d'écran.
- Fonctionnalités supplémentaires :
    - Notifications en temps réel pour informer les utilisateurs des changements critiques, comme la disponibilité ou la baisse des stocks de sang.

    - Accès rapide aux fonctions essentielles, comme l'enregistrement des dons, la consultation des stocks ou la planification des transfusions.
- Synchronisation et compatibilité : Assurer que la version mobile est parfaitement synchronisée avec le backend et la version web, afin que toutes les données soient cohérentes et accessibles en temps réel.
- Optimisation des performances : Réduire les temps de chargement et les consommations de ressources pour que l'application mobile reste fluide même sur des appareils moins performants ou avec une connexion internet faible.

### 3.3. Perspectives supplémentaires

- Évolution du système : Ces améliorations permettront de rendre LifeFlow plus robuste et sécurisé tout en facilitant l'adoption par les utilisateurs sur mobile.
- Maintenance et mise à jour : Les nouvelles fonctionnalités et optimisations doivent être accompagnées d'une documentation claire et d'un suivi régulier pour garantir la pérennité du projet.
- Évolutivité future : Une base solide sur la sécurité et la version mobile facilitera l'intégration de nouvelles fonctionnalités, comme des outils d'analyse, des rapports avancés et des alertes intelligentes pour la gestion des stocks de sang.

# Conclusion générale

En conclusion, le projet LifeFlow s'inscrit dans une démarche de modernisation et de digitalisation du système de gestion des dons de sang, en apportant une réponse concrète aux problématiques de traçabilité, de gestion des stocks et de coordination entre les différents acteurs du domaine médical. À travers ce travail, nous avons conçu et développé une plateforme complète, capable de centraliser les informations, d'automatiser les processus et d'améliorer significativement la réactivité face aux situations critiques.

L'adoption de la méthodologie Agile Scrum a joué un rôle essentiel dans la réussite du projet, en nous permettant d'organiser efficacement le travail, de prioriser les fonctionnalités et d'assurer une progression continue par itérations. Cette approche nous a également permis de mieux gérer les imprévus et d'améliorer la qualité du produit final grâce à des cycles réguliers de tests et de validation.

Sur le plan technique, le choix d'une architecture microservices, associé à l'utilisation de technologies modernes telles que Spring Boot, Flutter et des API REST sécurisées par JWT, a permis de concevoir une application robuste, évolutive et sécurisée. Cette architecture garantit une meilleure maintenabilité du système, une scalabilité adaptée aux besoins réels et une isolation efficace des différents modules.

Par ailleurs, ce projet nous a permis de consolider nos compétences en développement full stack, en conception UML, en gestion de projet et en cybersécurité. Il nous a également confrontés à des défis réels, notamment la complexité de la communication entre microservices, la gestion des rôles et des permissions, ainsi que l'optimisation des performances. Ces défis ont été surmontés grâce à une bonne collaboration au sein de l'équipe, une organisation rigoureuse et une démarche de résolution progressive des problèmes.

Les résultats obtenus démontrent que la solution proposée répond aux exigences fonctionnelles et non fonctionnelles définies initialement. L'application permet une gestion efficace des donneurs, des stocks sanguins, des demandes de transfusion et des campagnes de sensibilisation, tout en offrant une expérience utilisateur fluide et adaptée à chaque profil.

Enfin, plusieurs perspectives d'évolution peuvent être envisagées afin d'enrichir davantage la plateforme, telles que l'intégration de l'intelligence artificielle pour la prédiction des

besoins en sang, l'ajout de fonctionnalités de géolocalisation pour faciliter la mise en relation entre donneurs et hôpitaux, ou encore le déploiement sur une infrastructure cloud pour assurer une disponibilité à grande échelle.

**Au-delà d'un simple projet informatique, LifeFlow incarne notre volonté d'utiliser la technologie pour répondre à un enjeu vital : sauver des vies.**

# Bibliographie


- https://fr.scribd.com/document/789586926/METHODE-AGILE-SCRUM
- https://cyber.gouv.fr/securisation/analyse-des-risques/methode-ebios-rm/
- https://www.c-risk.com/fr/blog/iso-27005
- https://cloud.google.com/learn/what-is-microservices-architecture?hl=fr

# Annexes

# Liste des figures des annexes

Figure 1 : Test de sécurité (ZAP).....84

Figure 2 : Interface Notifications Mobile.....84

Figure 3: Interface Chatbot Mobile.....85

Figure 4 : Interface Profile Mobile.....85

Figure 5 : Interface Des Dons Mobile.....86

Figure 6: Interface de la création du compte Mobile.....86

Figure 7 : Interface Accueil Mobile.....87

Figure 8 : Interface du classement Mobile….....87

Figure 9: Interface de la Modification du profil Mobile.....88

Figure 10 : Interface des Badges Mobile.....88

Figure 11 : Interface de connexion Mobile.....89

Figure 12: Interface de déclaration des dons Mobile.....89

Figure 13 : Interface de gestion des donneurs Web.....90

Figure 14 : Interface de gestion des dons Web.....90

Figure 15: Interface du profil donneur Web.....91

Figure 16 : Interface du site avant la connexion Web.....92

Figure 17 : Interface de gestion de stock Web.....92

Figure 18: Interface Gestion des hôpitaux Web.....93

Figure 19 : Interface Gestion de stock Web.....93

Figure 20 : Interface de gestion des campagnes Web.....94

Figure 21: Interface de notifications globales.....94

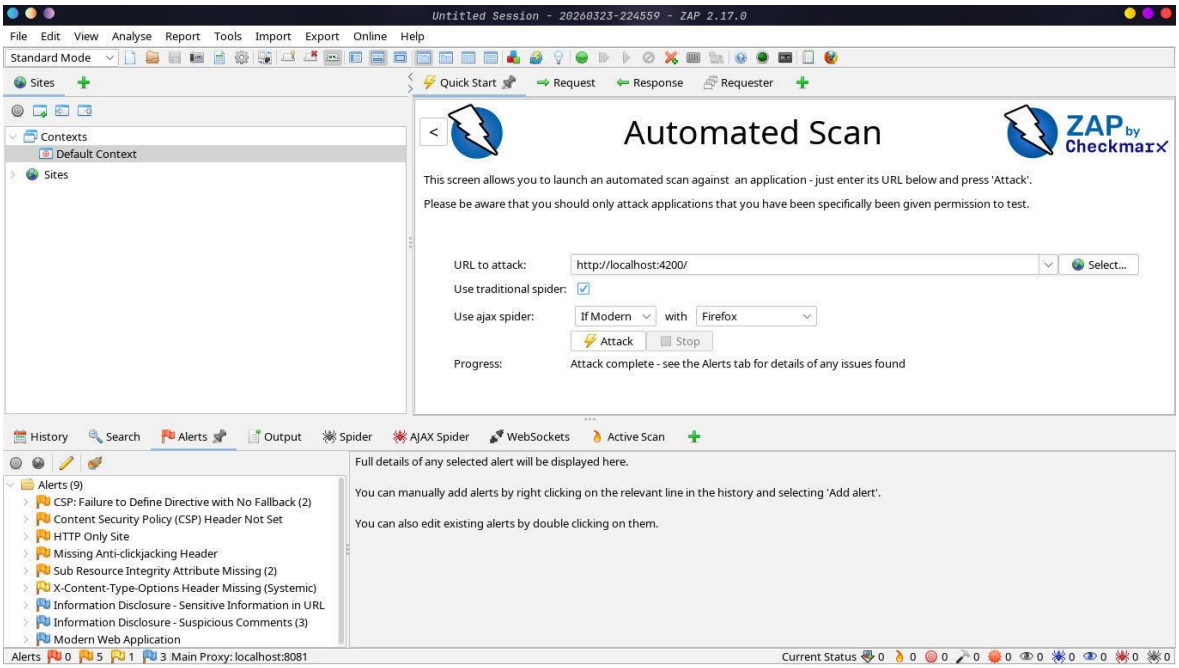
Figure 22 : Interface de gestion de sang Web.....95

Figure 23: Interface des statistiques globales du système Web.....95

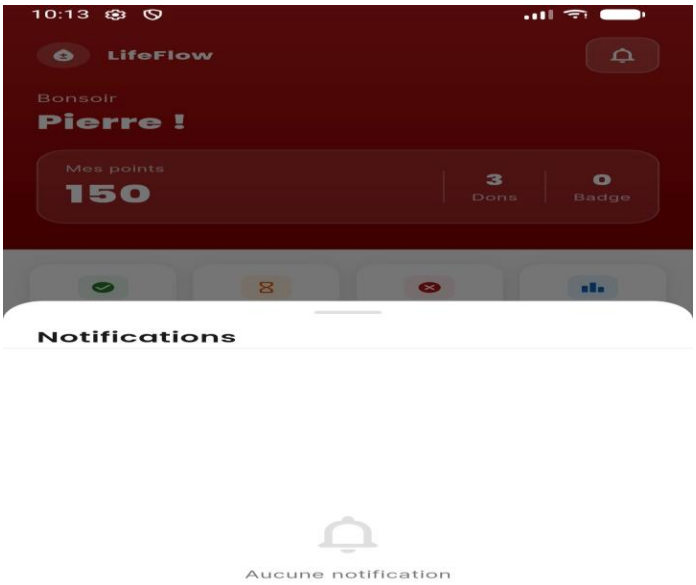
Figure 24 : Interface des campagnes nationales Web.....96

Figure 25 : Interface des tests de Postman.....96

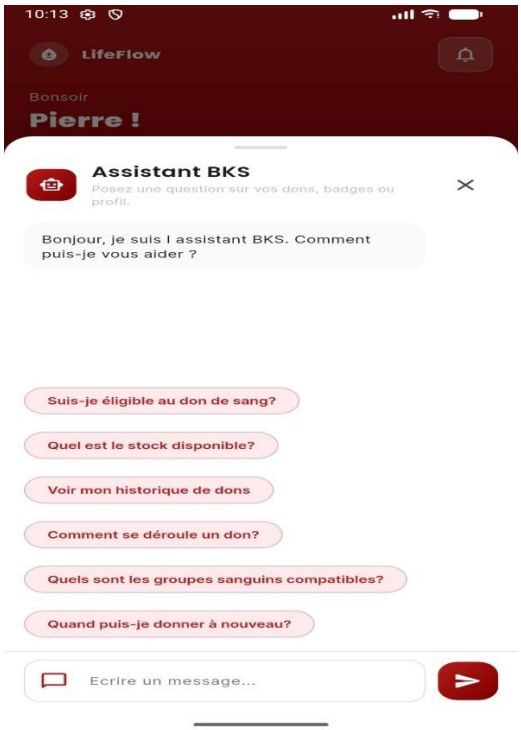
Figure 26 : Interface des tests de Postman.....97



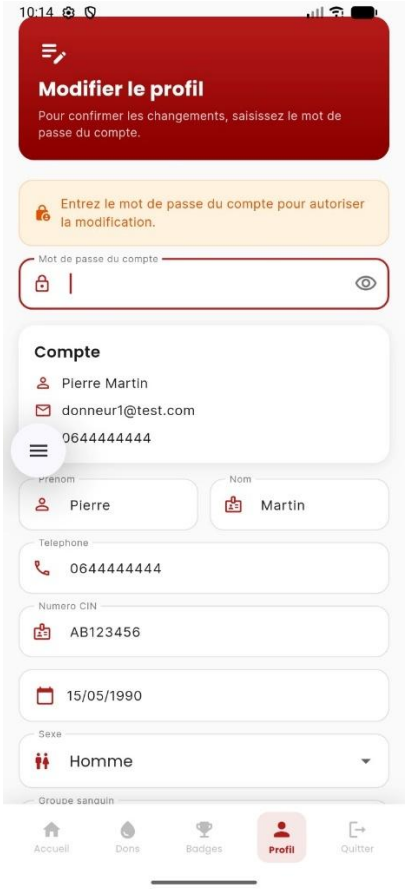

**Test de sécurité (ZAP)**




**Interface Notifications Mobile**

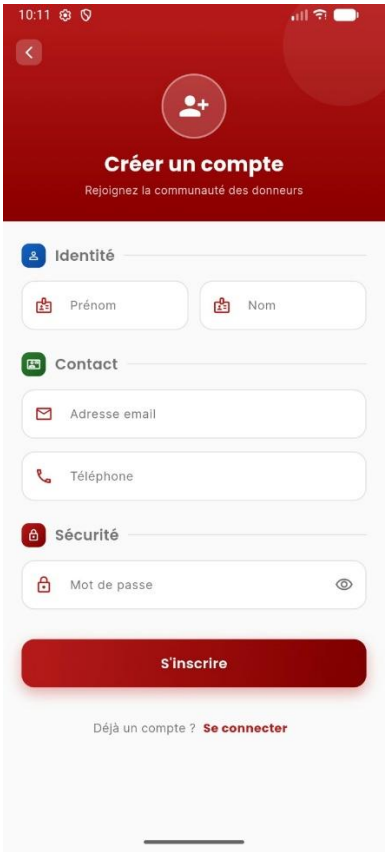



**Interface Chatbot Mobile**

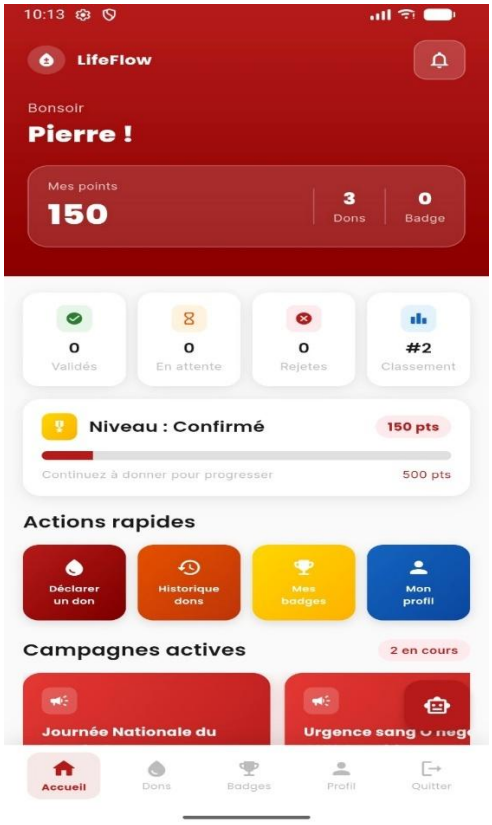



**Interface Profile Mobile**

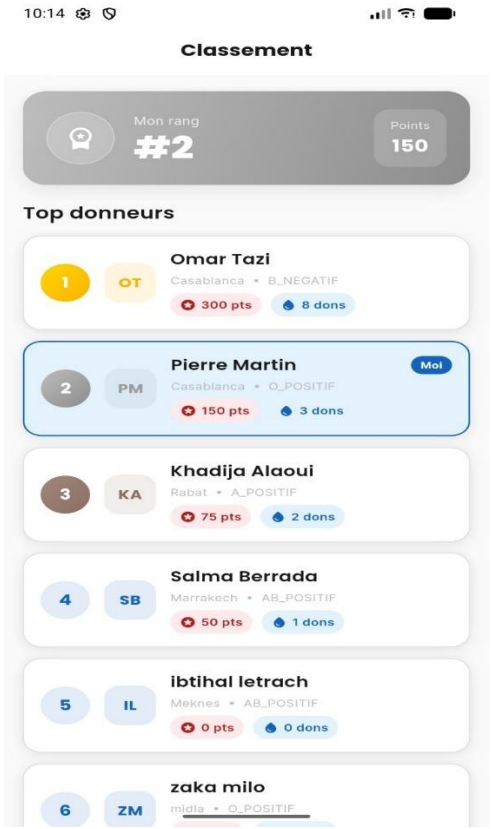
**Interface Des Dons Mobile**



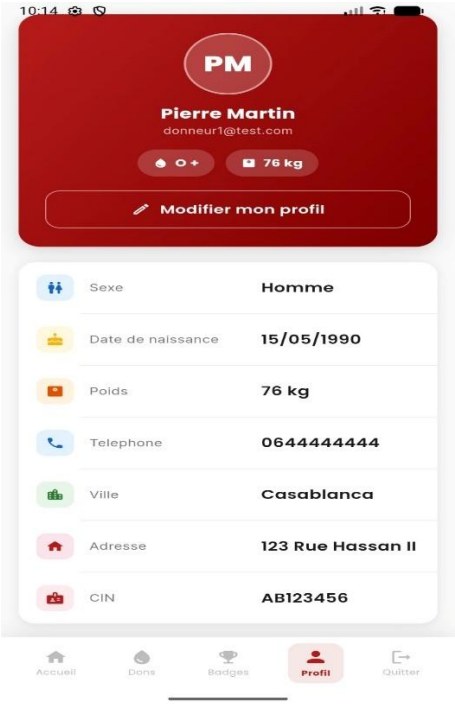

**Interface de la création du compte Mobile**




**Interface Accueil Mobile**



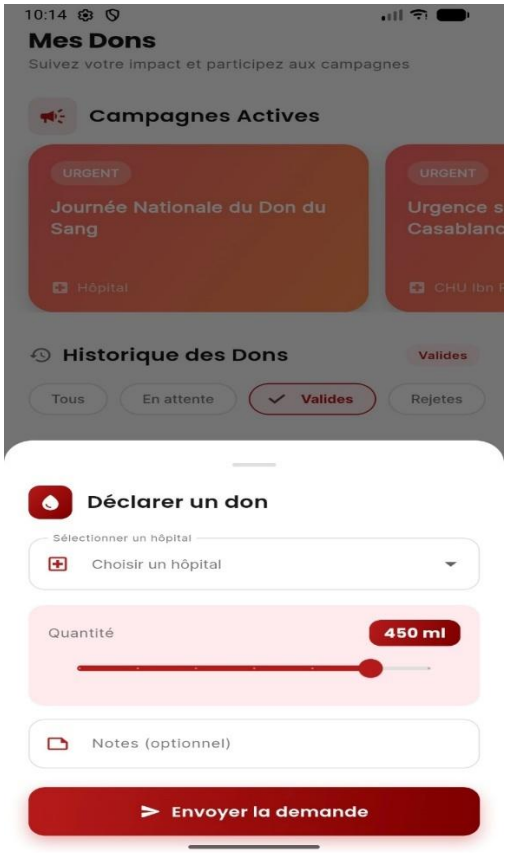
**Interface du classement Mobile**



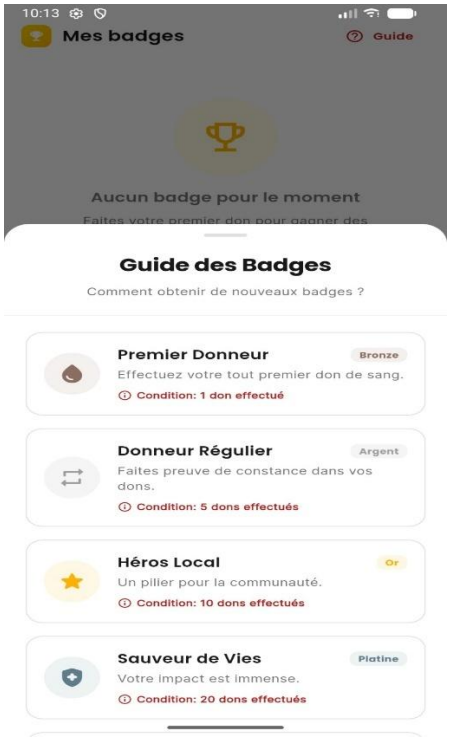

**Interface de la Modification du profil Mobile**

**Interface des Badges Mobile**

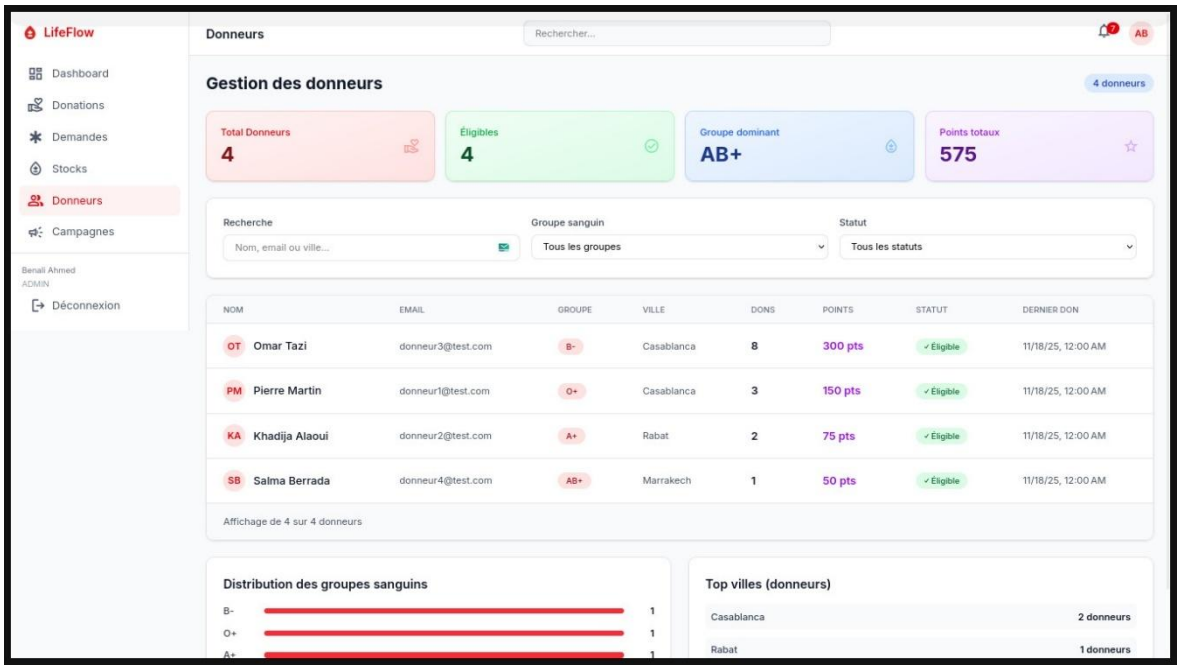
**Interface de connexion Mobile**



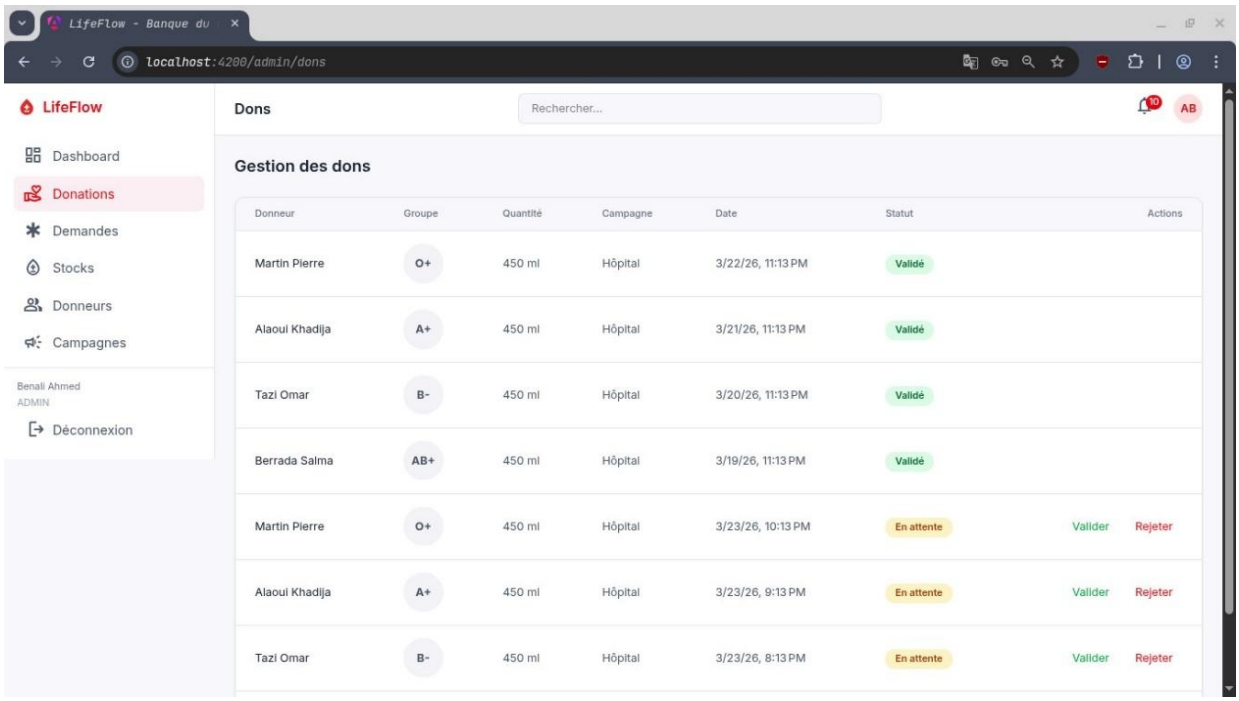
**Interface de déclaration des dons Mobile**



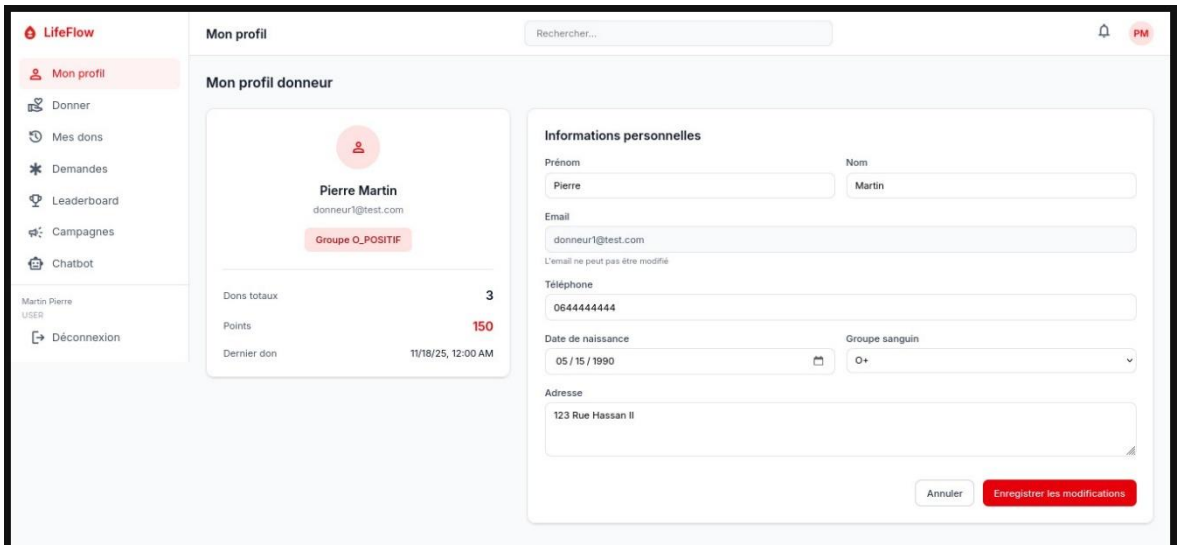

**Interface de guide des badges Mobile**



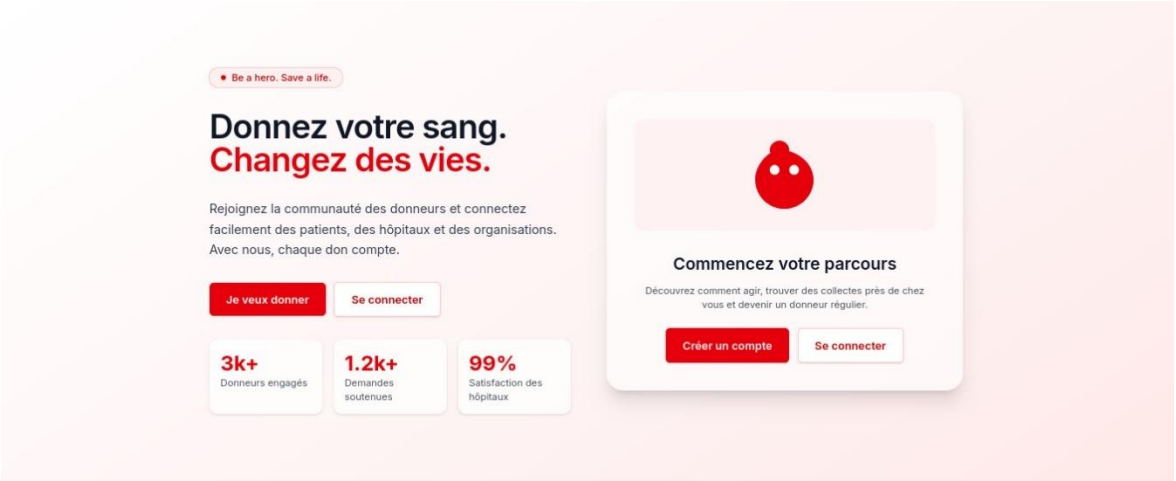

**Interface de gestion des donneurs Web**



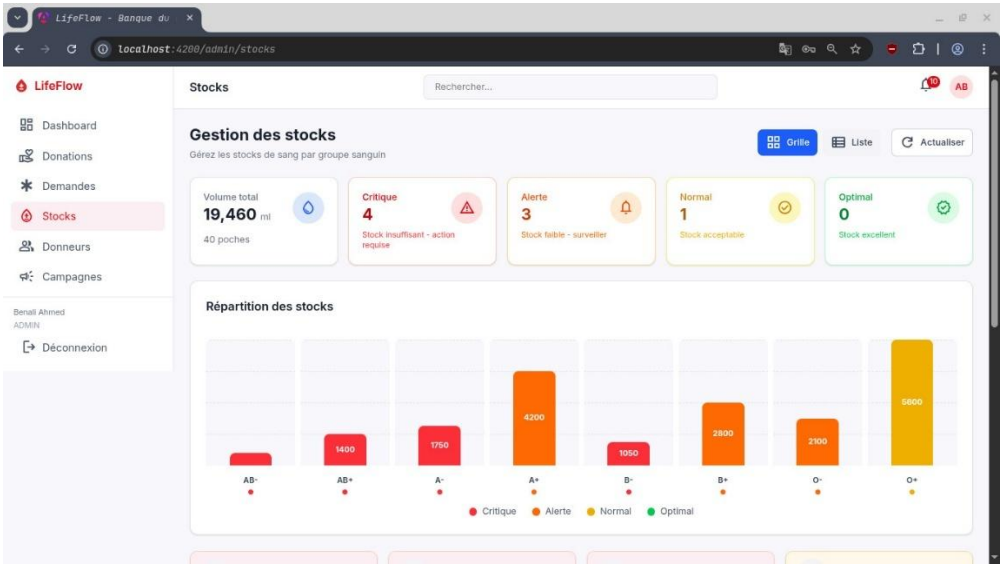

**Interface de gestion des dons Web**



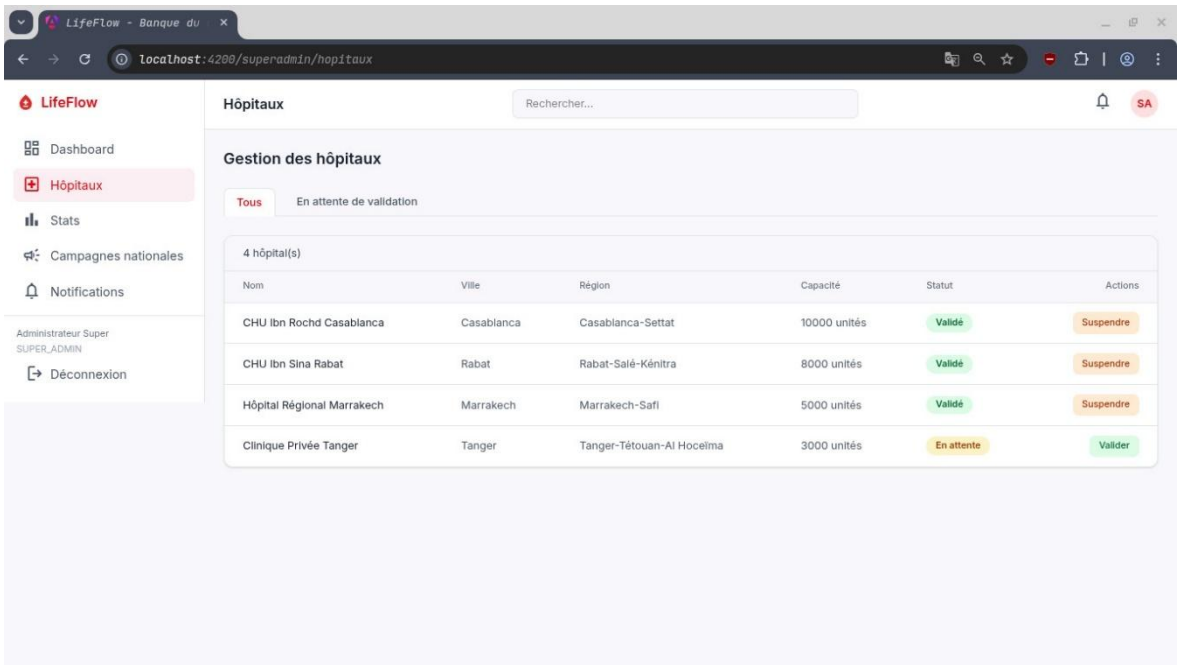

**Interface du profil donneur Web**



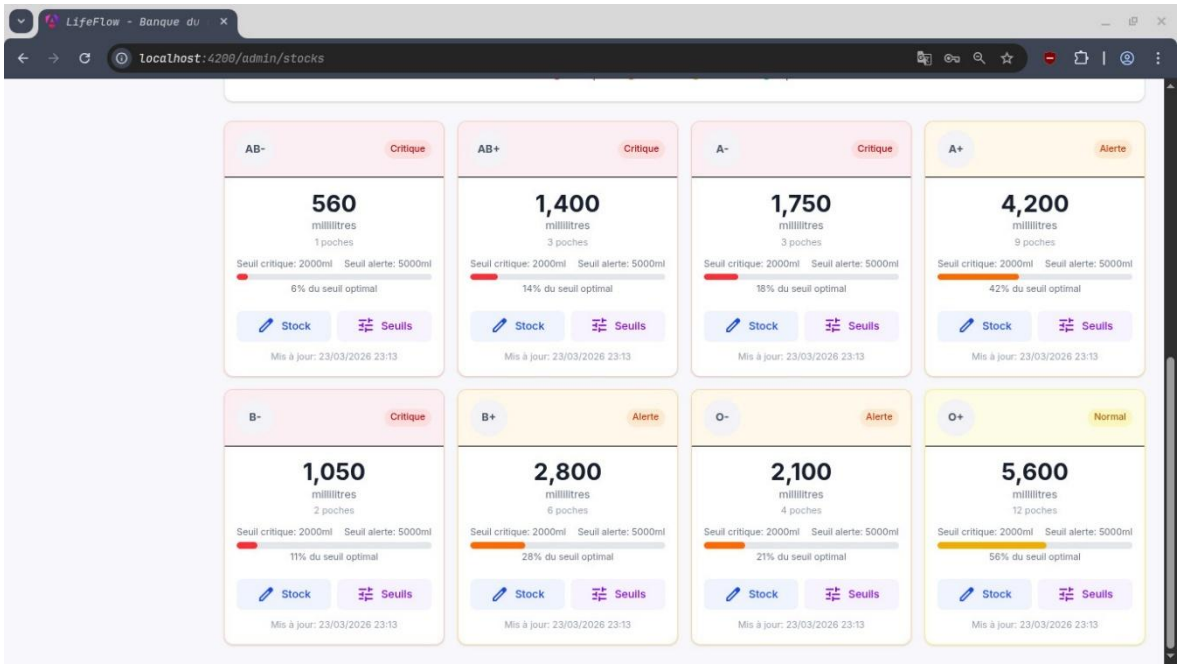

**Interface du site avant la connexion Web**



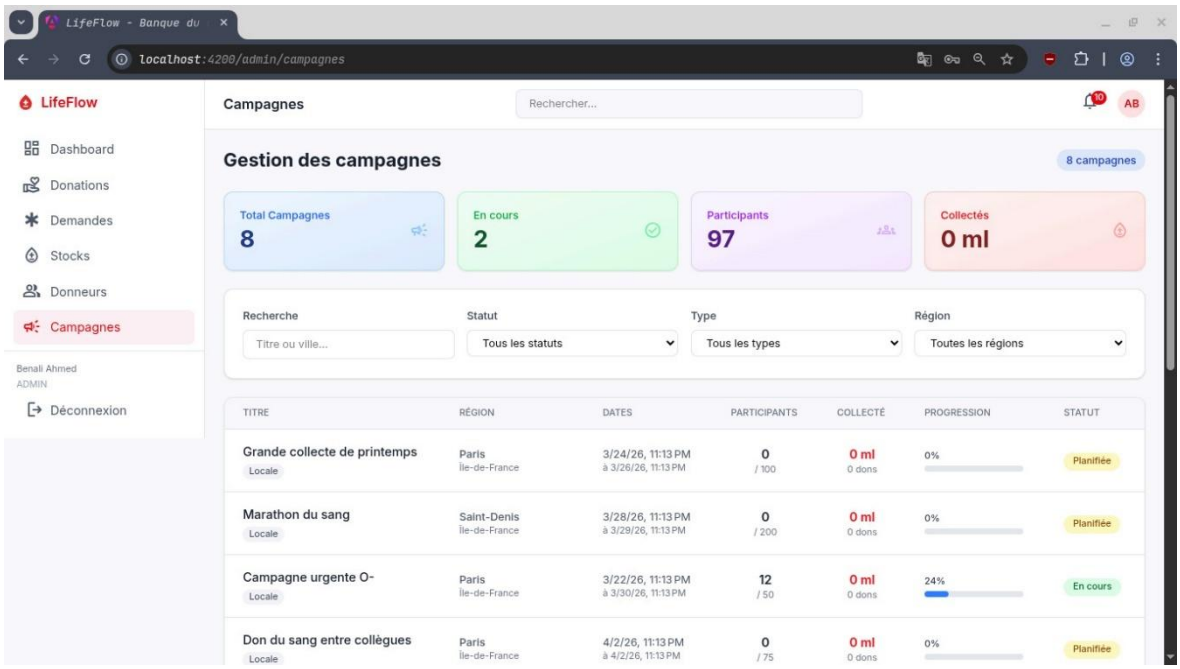

**Interface de gestion de stock Web**



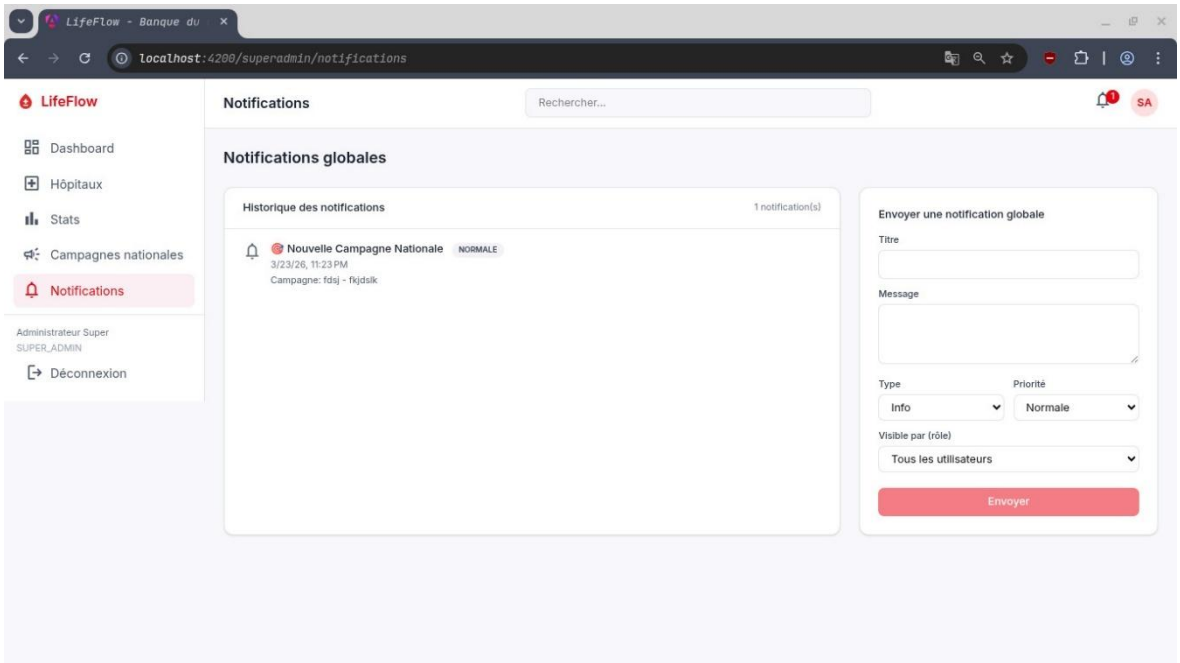

**Interface Gestion des hôpitaux Web**



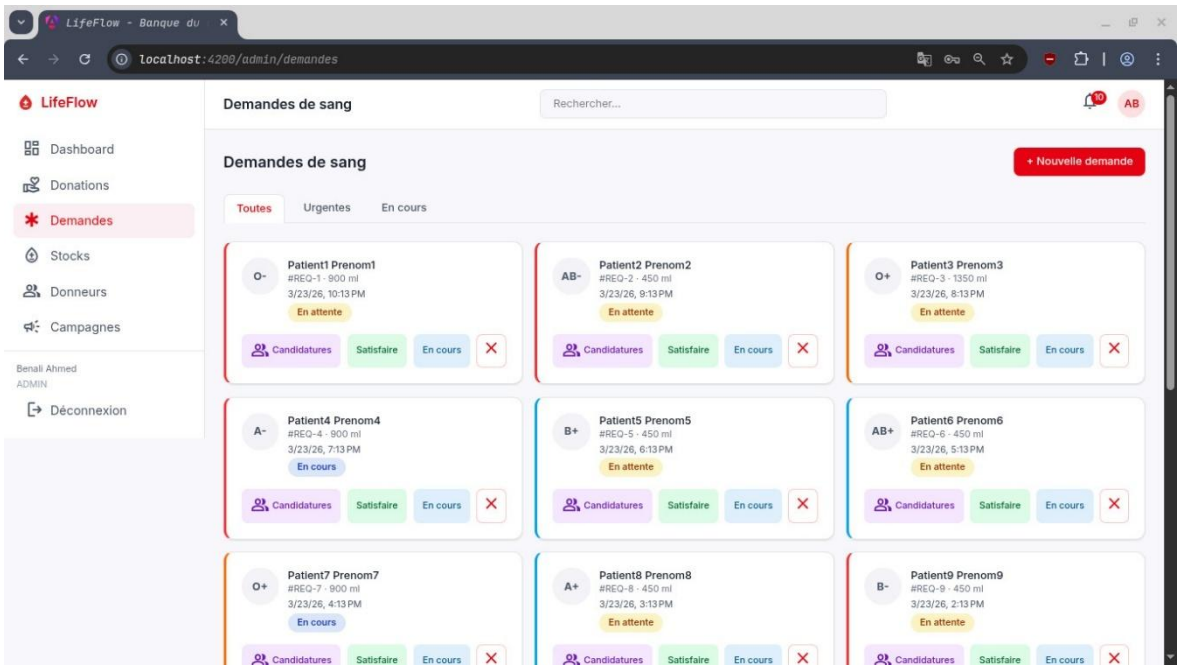

**Interface Gestion de stock Web**



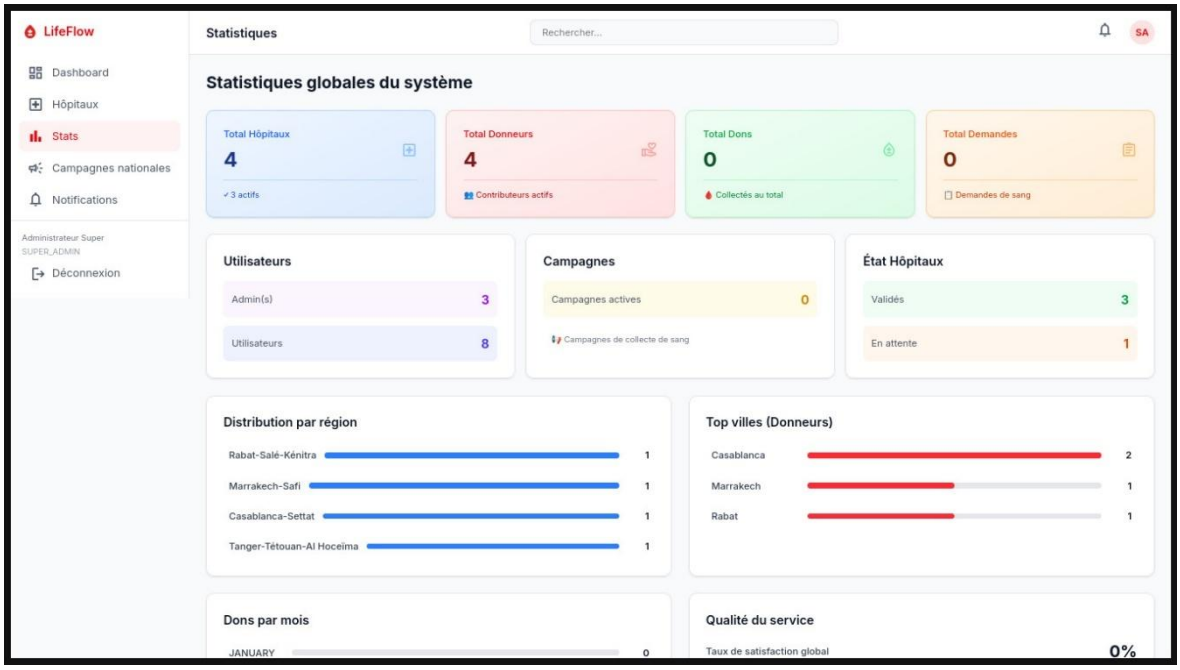

**Interface de gestion des campagnes Web**



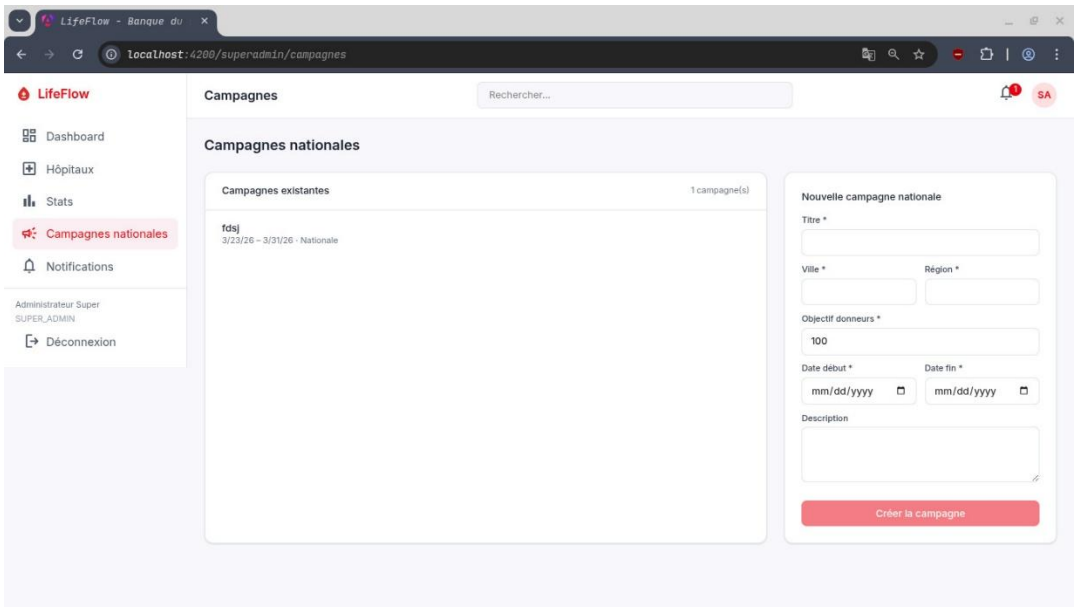

**Interface de notifications globales**



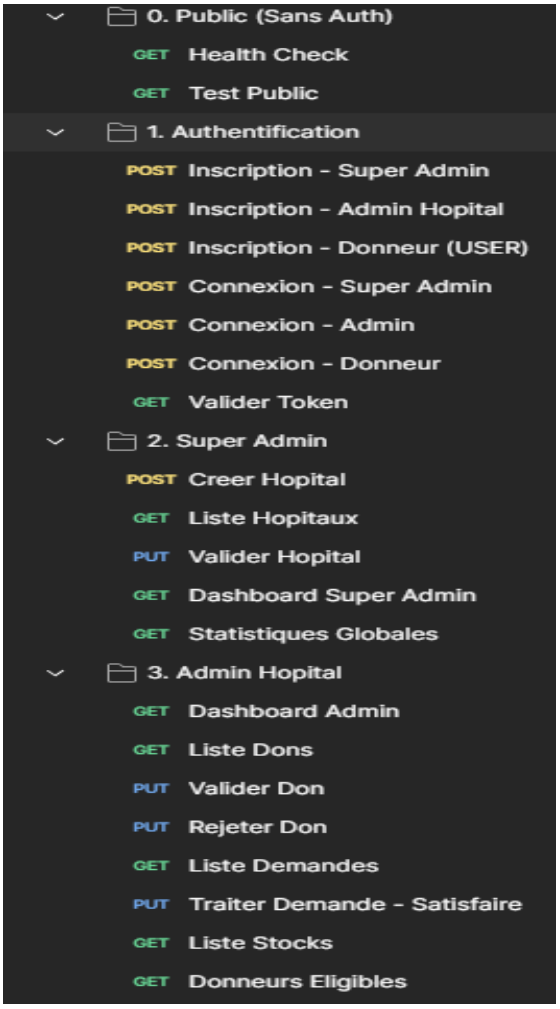

**Interface de gestion de sang Web**



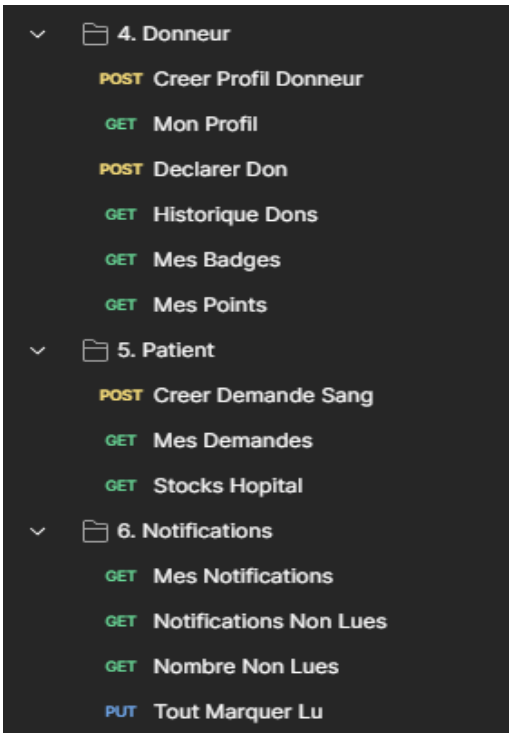

**Interface des statistiques globales du système Web**




**Interface des campagnes nationales Web**




**Interface des tests de Postman**




**Interface des tests de Postman**

**Mobile :**

https://github.com/LifeFloww/mobile.git

**Web :**

https://github.com/zakaryaemiloudy/web.git